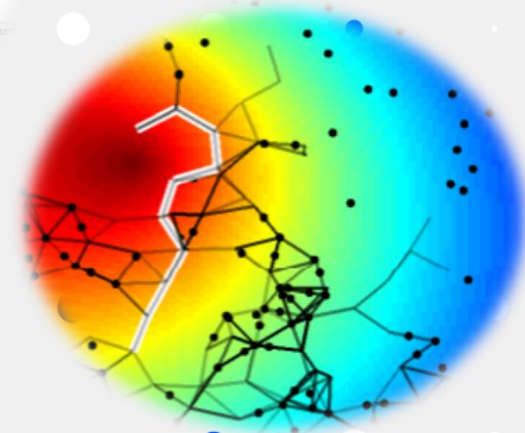
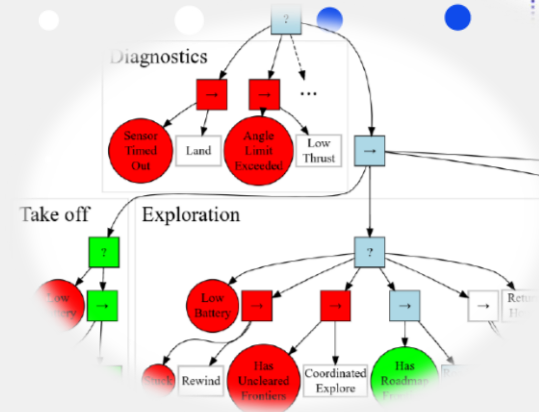
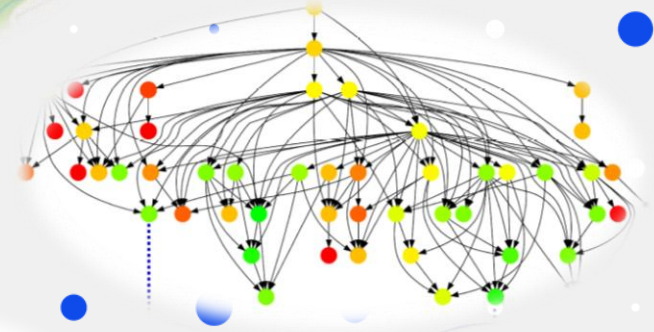
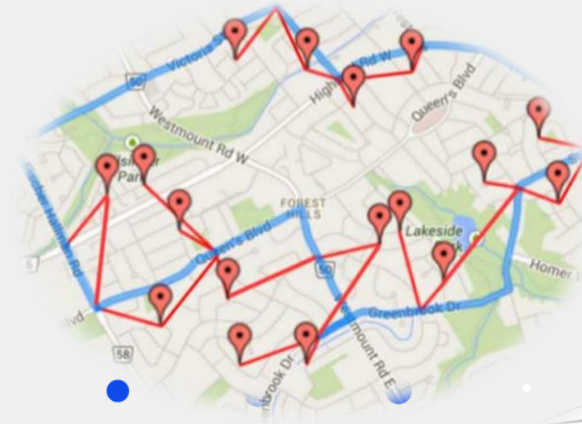


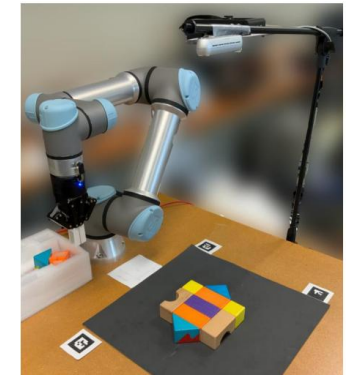
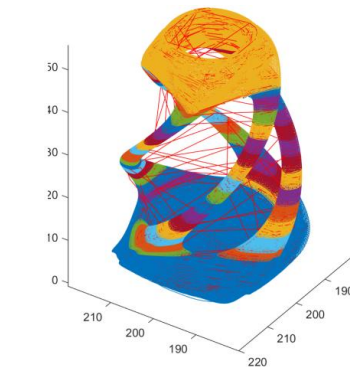
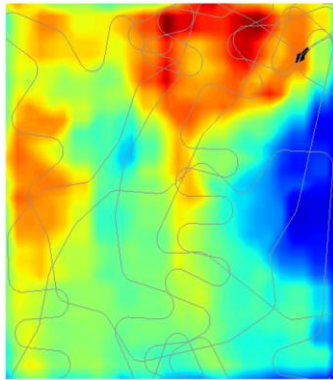
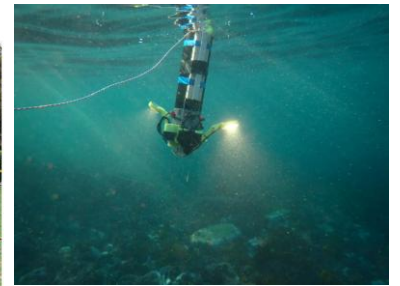
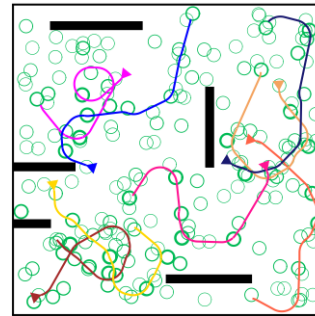
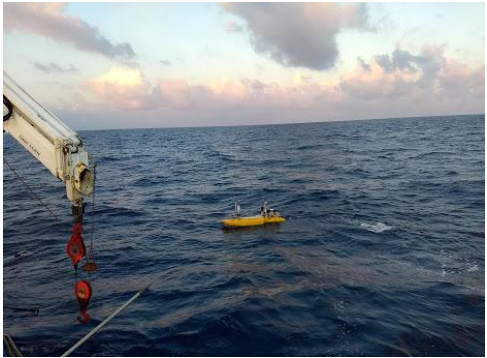


Decision Making

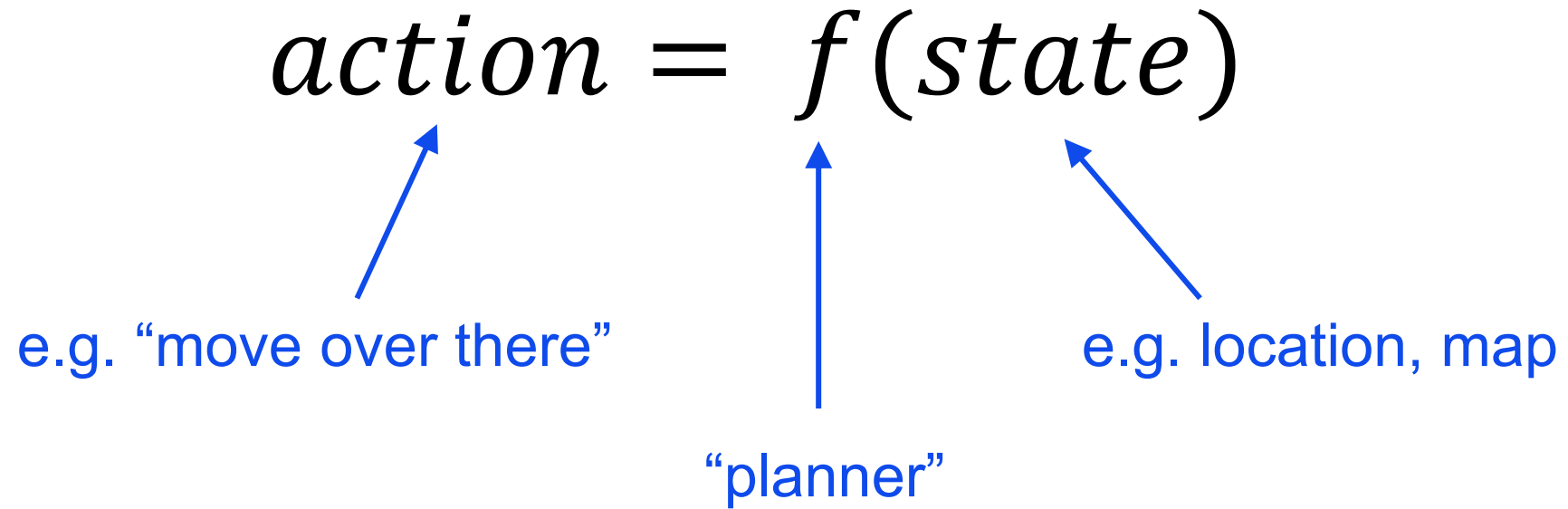
Graeme Best
41068 Robotics Studio 1

Faculty of Engineering and Information Technology
University of Technology Sydney

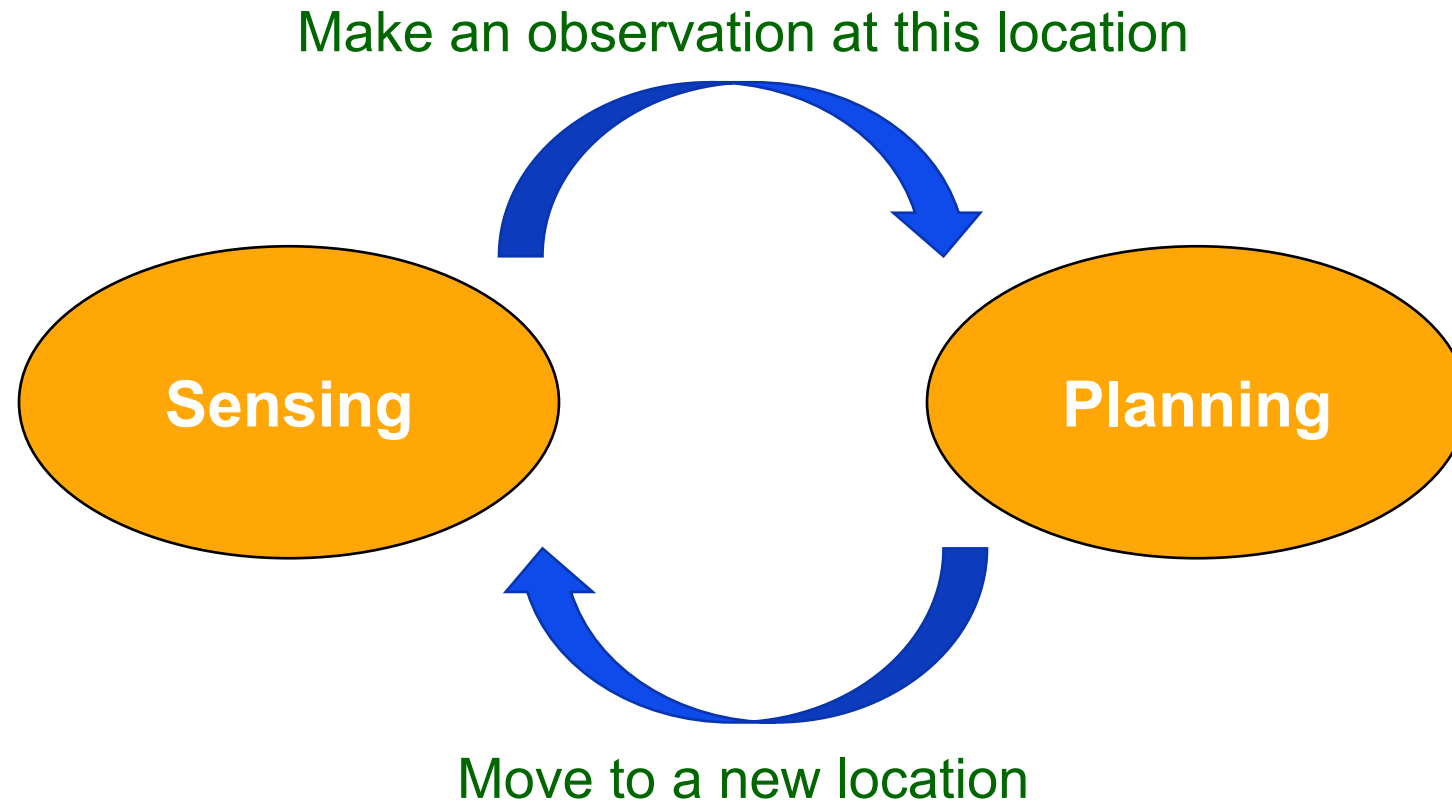




Robot Decision Making



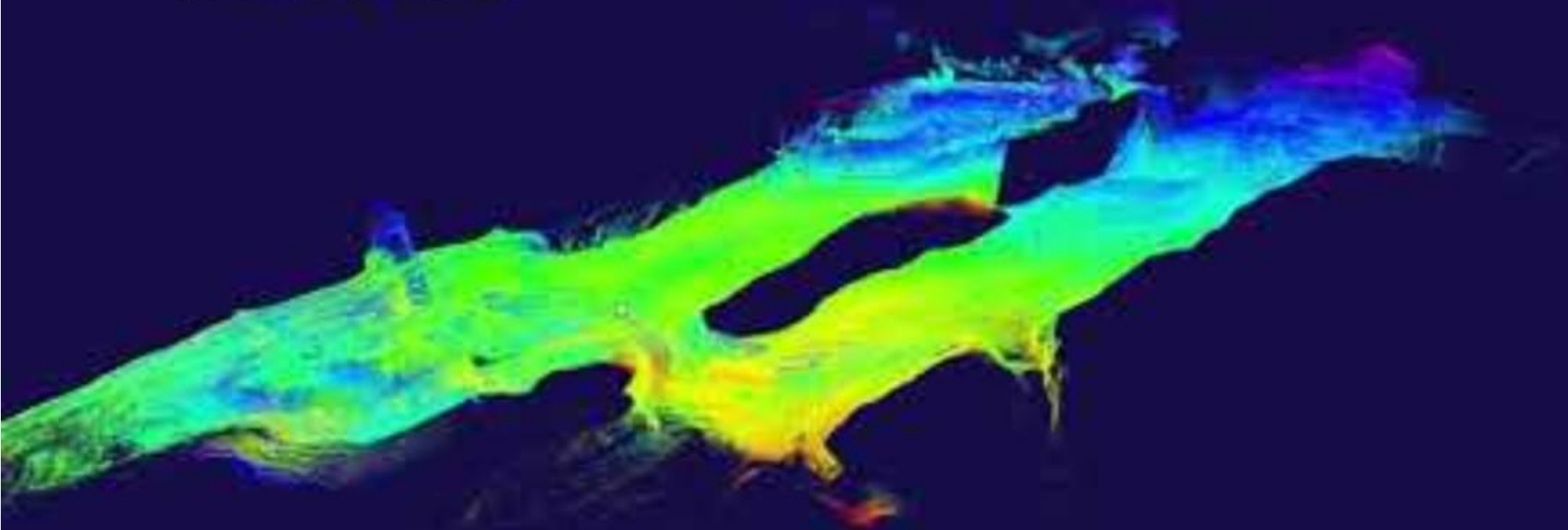
Coupled Sensing and Planning



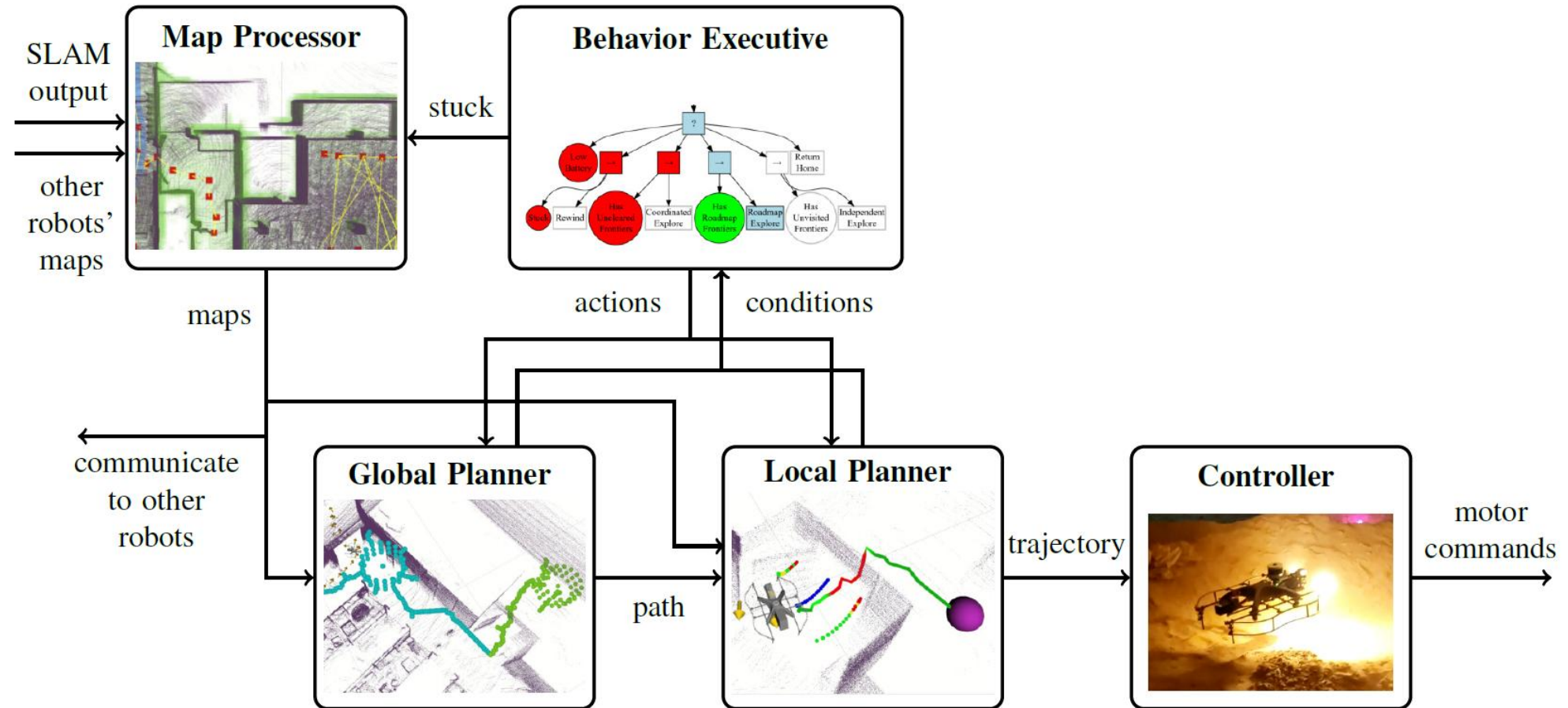


Laurel Caverns

Mapping



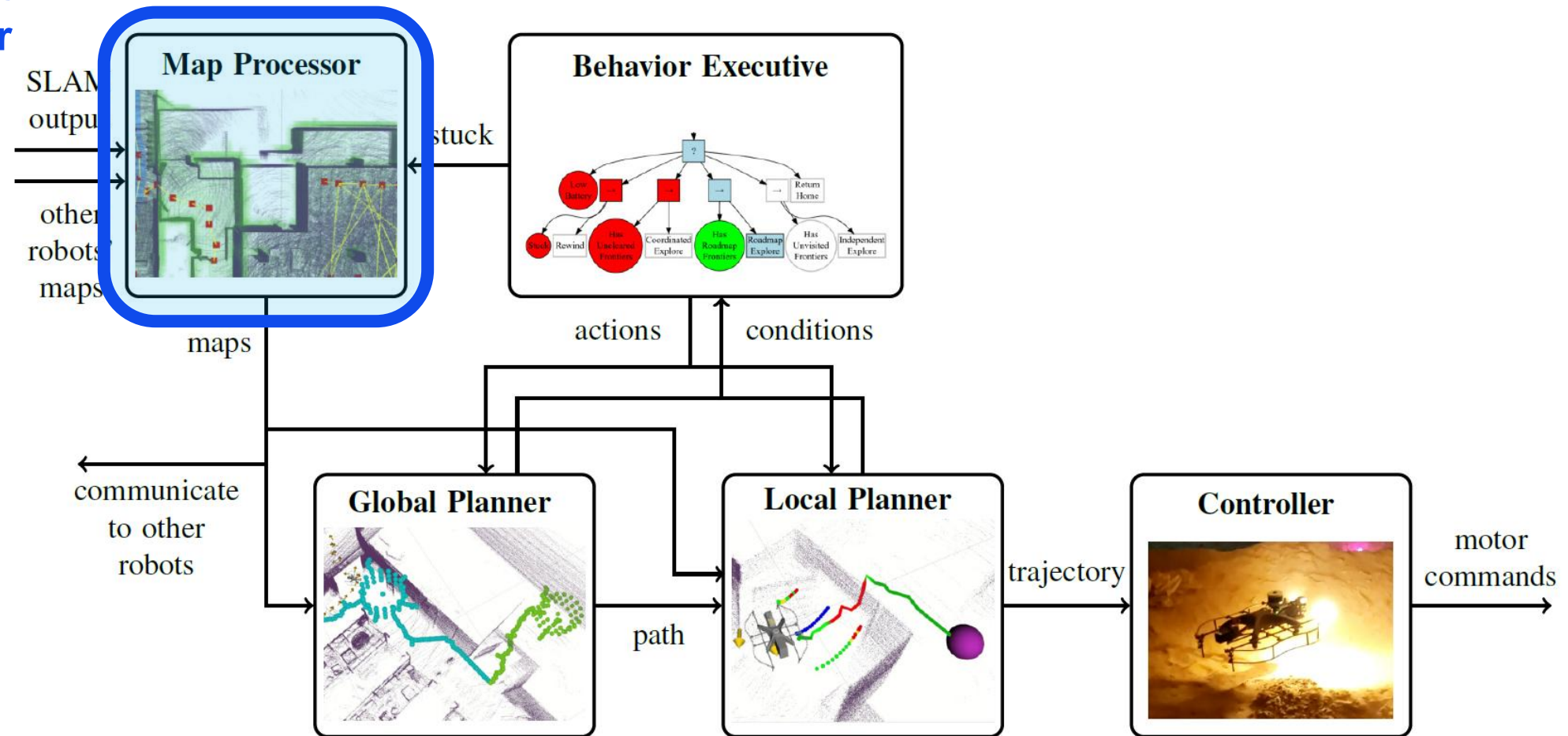
Example Decision Making Hierarchy



Example Decision Making Hierarchy



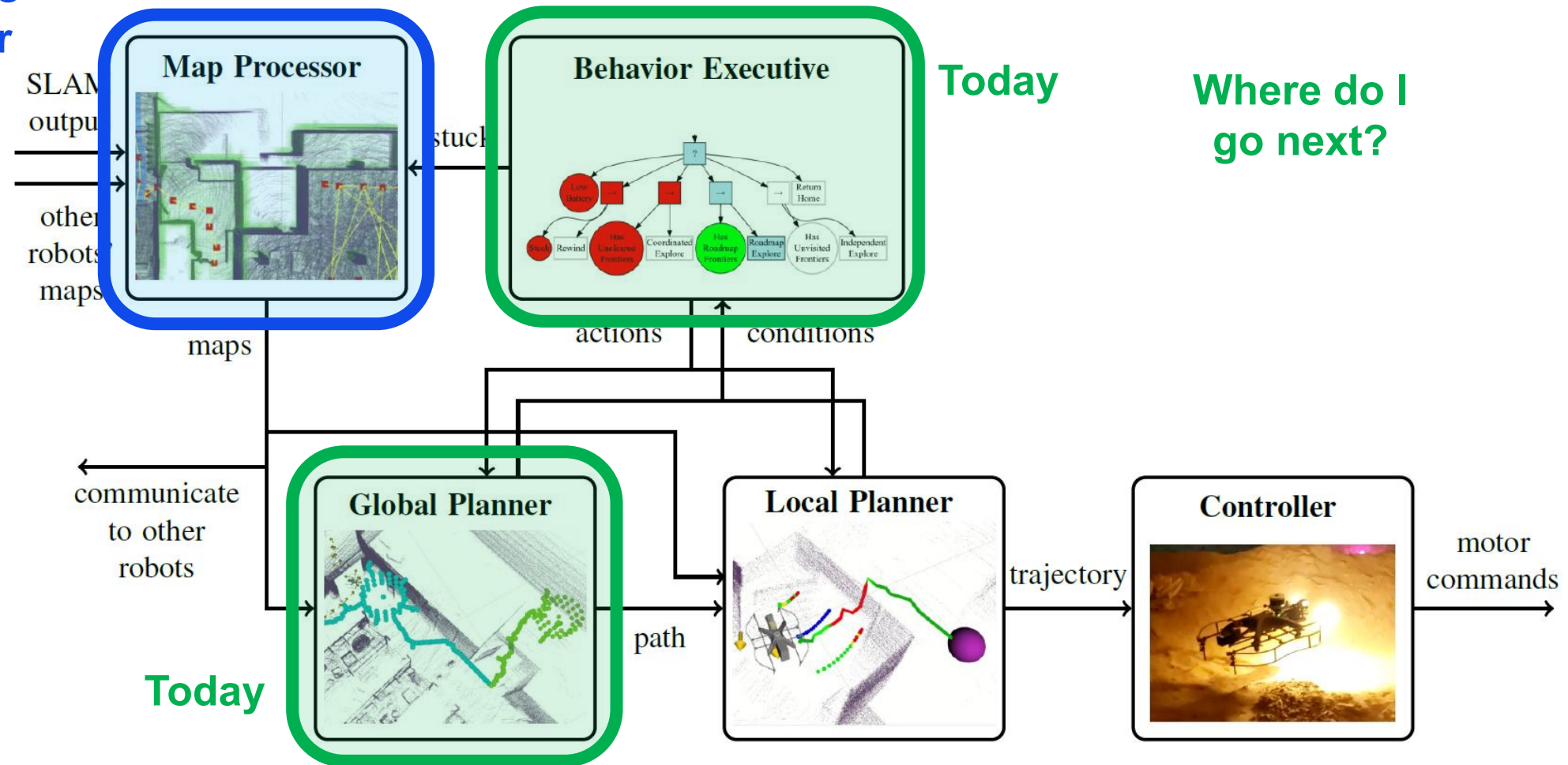
Previous
seminar



Example Decision Making Hierarchy



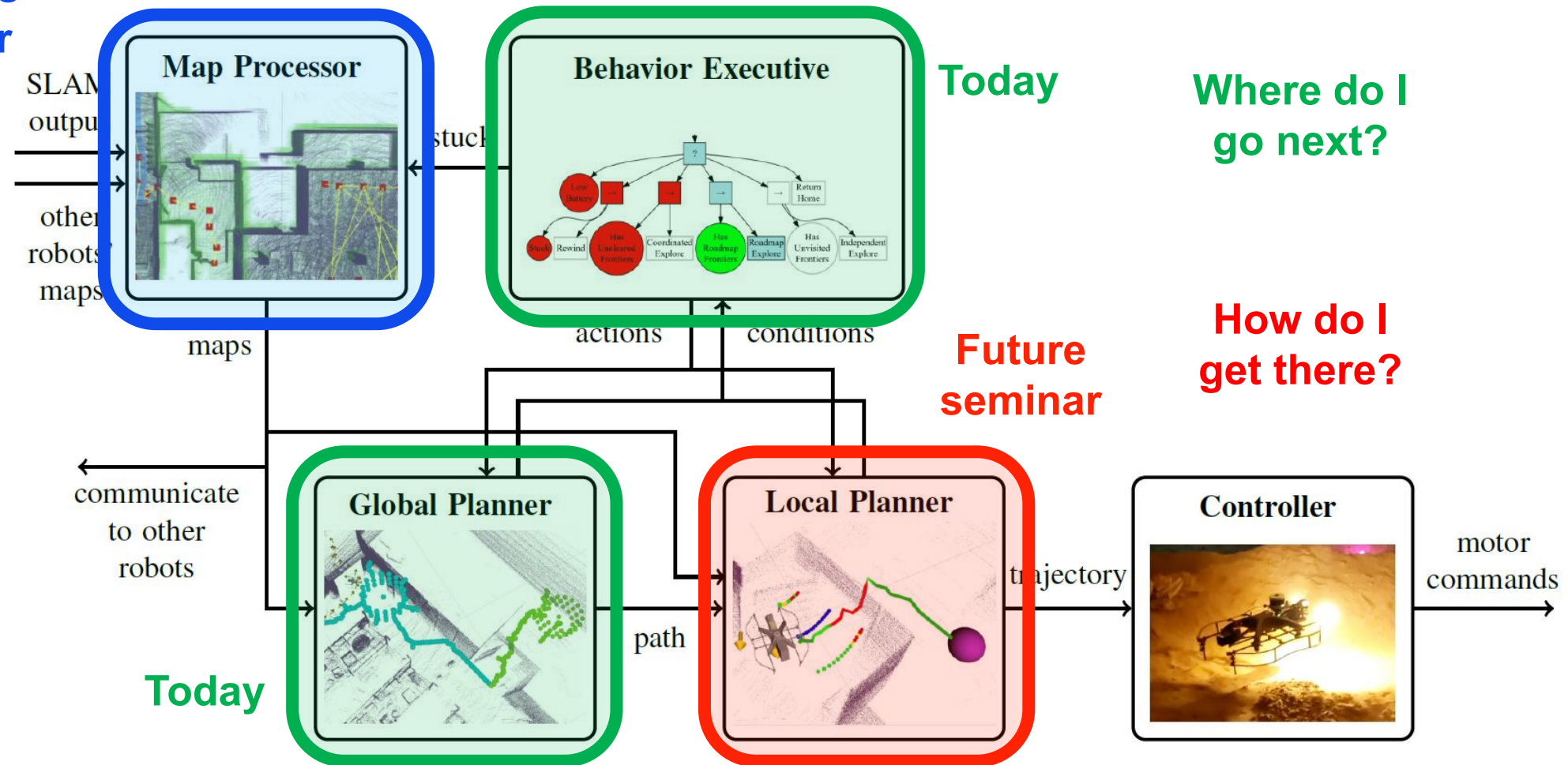
Previous
seminar



Example Decision Making Hierarchy



Previous
seminar



What is Decision Making?



- **Robot decision making** is the process of choosing *what the robot should do next* based on its current knowledge, goals, and environment.
- It includes selecting the next **task, goal, or behaviour**, rather than just how to execute a specific movement.
- Good decision-making helps a robot act **autonomously**, adapting to changing conditions, and make the best use of its time, sensors, and capabilities.

Why is Decision Making Challenging?



- Defining the right objective for the application
- Incomplete and uncertain information – usually from noisy sensors
- Interact with lower-level planners and controllers
- Computationally difficult – many options to pick from
- Choosing the right algorithm for the task
- Tuning algorithm parameters
- Multi-robot coordination

Questions for your 41068 Project



- Ideas for different types of decision making
- Your project must have some decision making
- How will your robot decide where to go next?
 - What is the objective?
 - What algorithms will solve this problem?
 - How will these algorithms connect to the rest of your system?



Outline

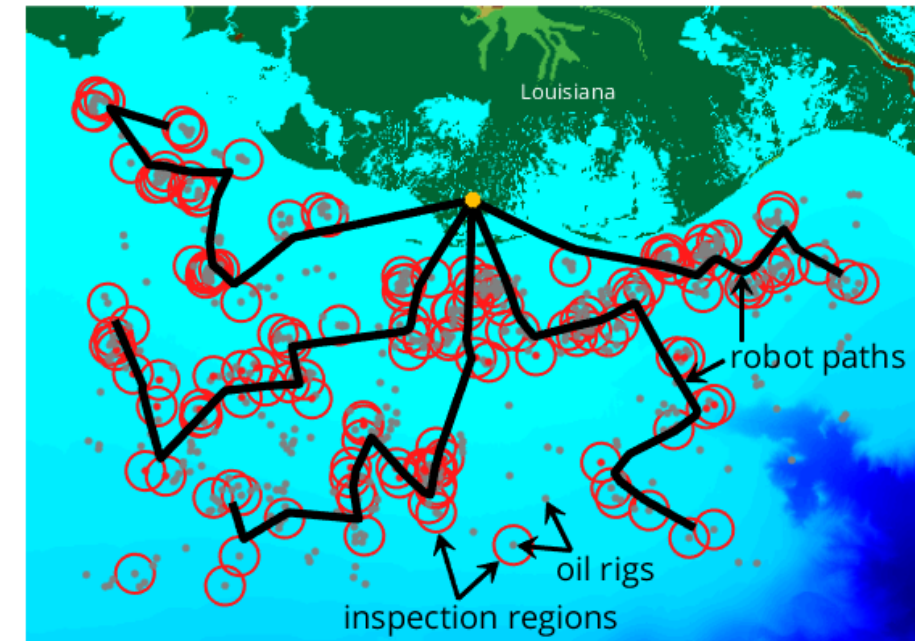
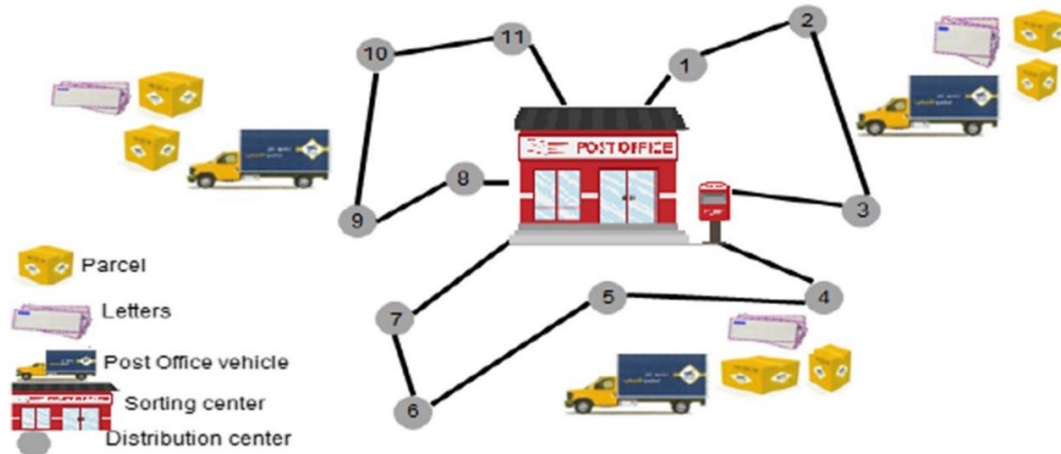
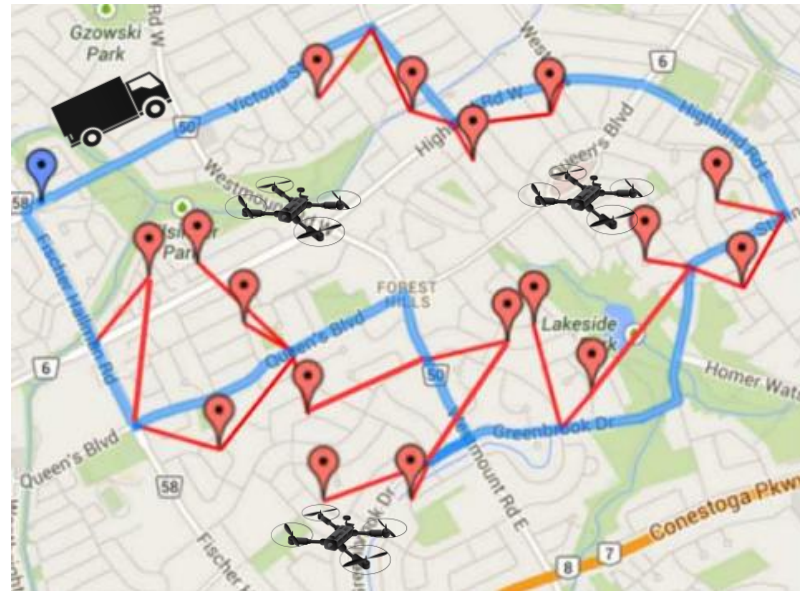


- Travelling salesman problem
- Exploration
- Informative path planning
- Monte Carlo tree search
- Behaviour trees



Travelling Salesman Problem

Vehicle Routing Problems

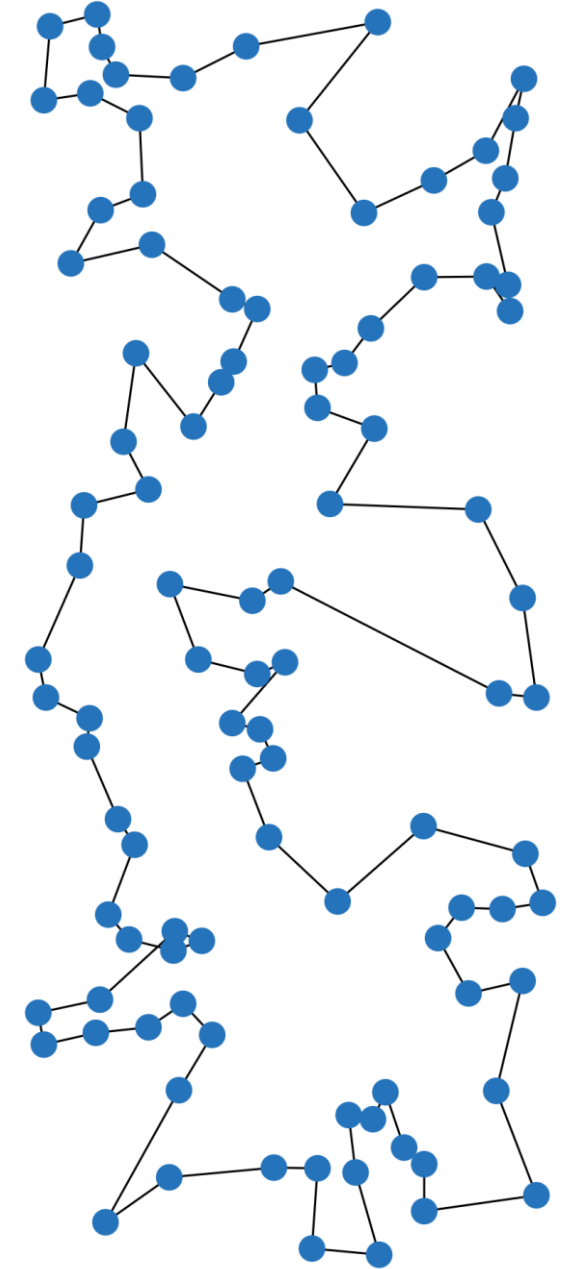


Travelling Salesman Problem

- Widely studied benchmark problem in Computer Science
- Useful approximation to some robotics problems
- Many solutions exist
- Many variants exist

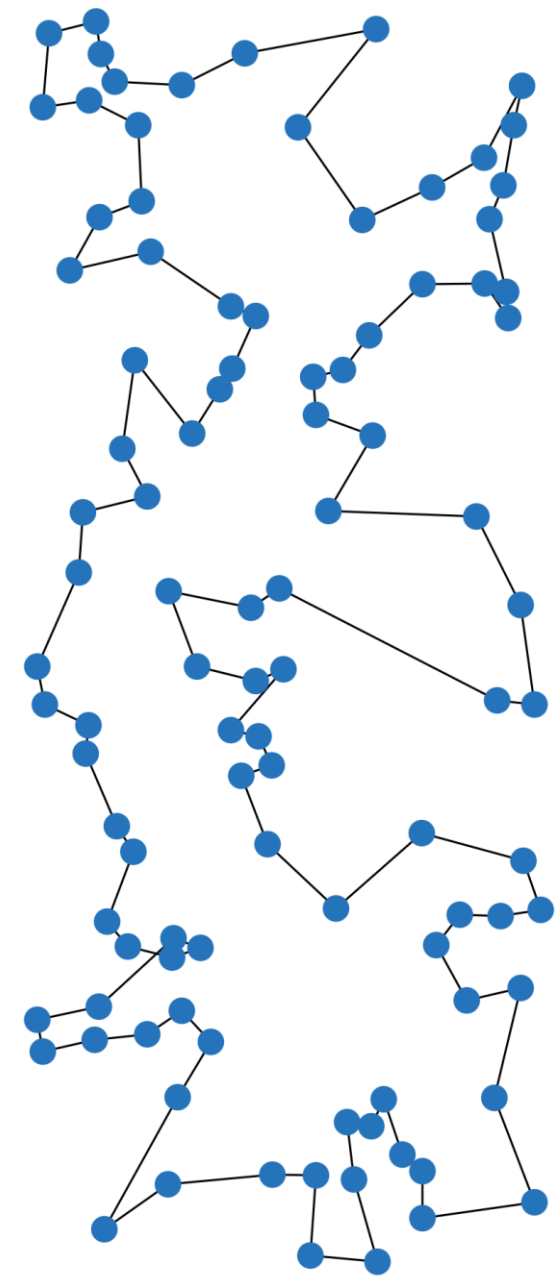
Problem:

Given a list of locations and the distances between locations, what is the shortest possible route that visits each location exactly once and returns to the origin?



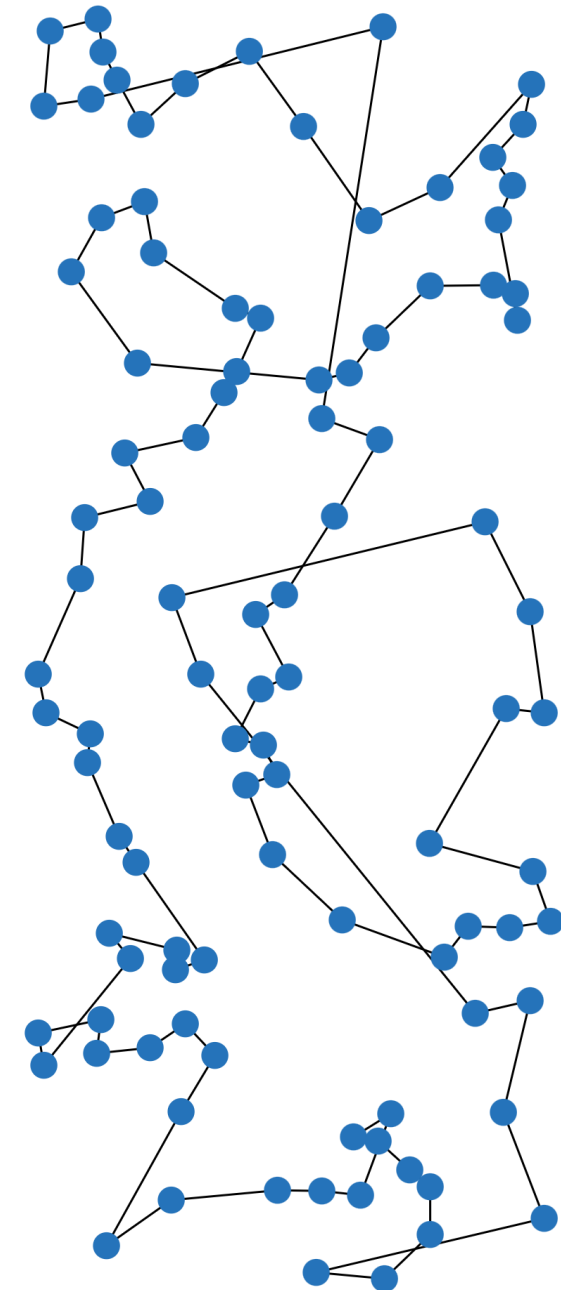
Solution: Brute-Force Search

- Optimal algorithm, very slow $O(n!)$ time
- 1. Arbitrarily label all of the nodes
- 2. Try all possible permutations
 - $(1,2,3, \dots, n)$
 - $(2,1,3, \dots, n)$
 - $(1,3,2, \dots, n)$
 - $(2,3,1, \dots, n)$
 - ...
- 3. Pick the permutation with the lowest cost



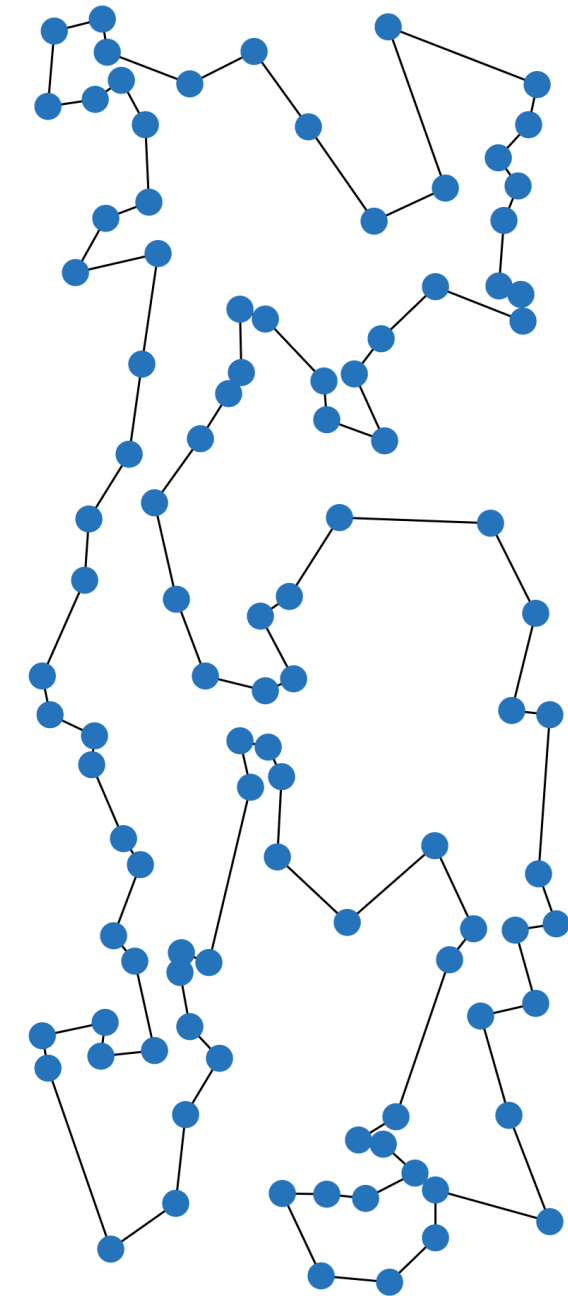
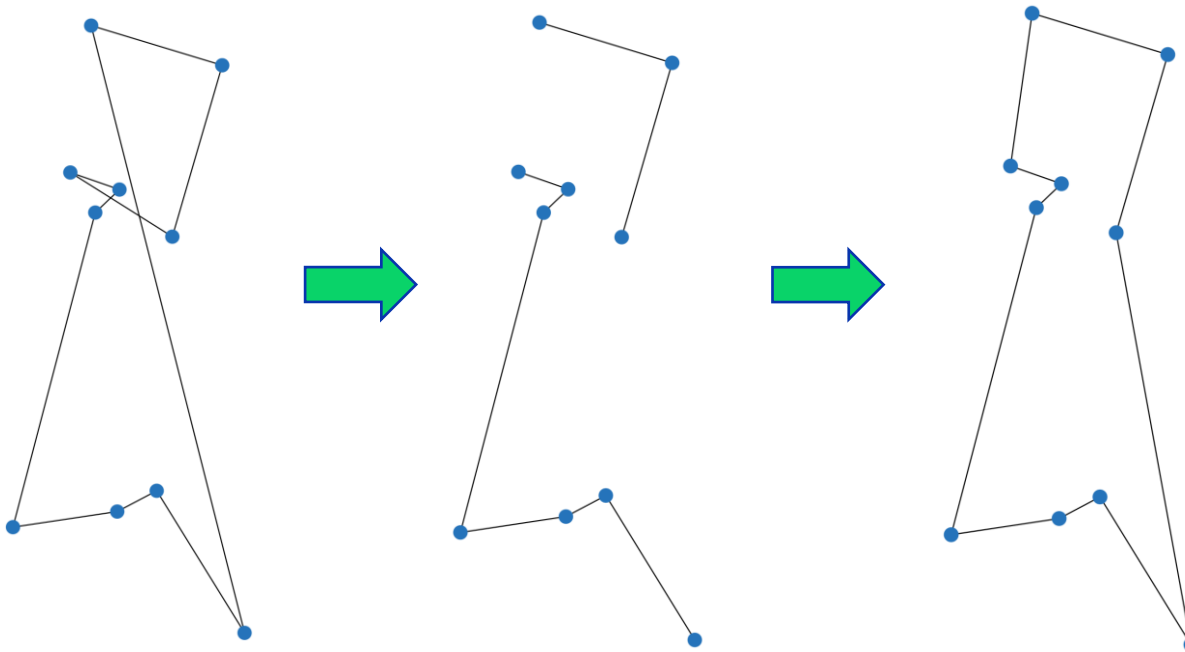
Solution: Nearest Neighbour

- A “greedy” algorithm
 - Making a locally-optimal choice at each stage
- 1. Pick a starting node
- 2. Move to the nearest neighbour that has not yet been visited
- 3. Repeat step 2



Solution: 2-Opt

1. Start with a nearest-neighbour solution
2. Pick two random edges in the existing solution
3. Replace them with a better pair of edges
4. Repeat 2 and 3



Off-the-Shelf Packages



- You could implement solutions yourself
- Many off-the-shelf packages exist,
e.g. python_tsp: https://pypi.org/project/python_tsp/

```
import numpy as np
from python_tsp.exact import solve_tsp_dynamic_programming

distance_matrix = np.array([
    [0, 5, 4, 10],
    [5, 0, 8, 5],
    [4, 8, 0, 3],
    [10, 5, 3, 0]
])
permutation, distance = solve_tsp_dynamic_programming(distance_matrix)
```

The solution will be [0, 1, 3, 2]

Define the distance between each pair of points.
Infinity means disconnected.

Visit the locations in this order.



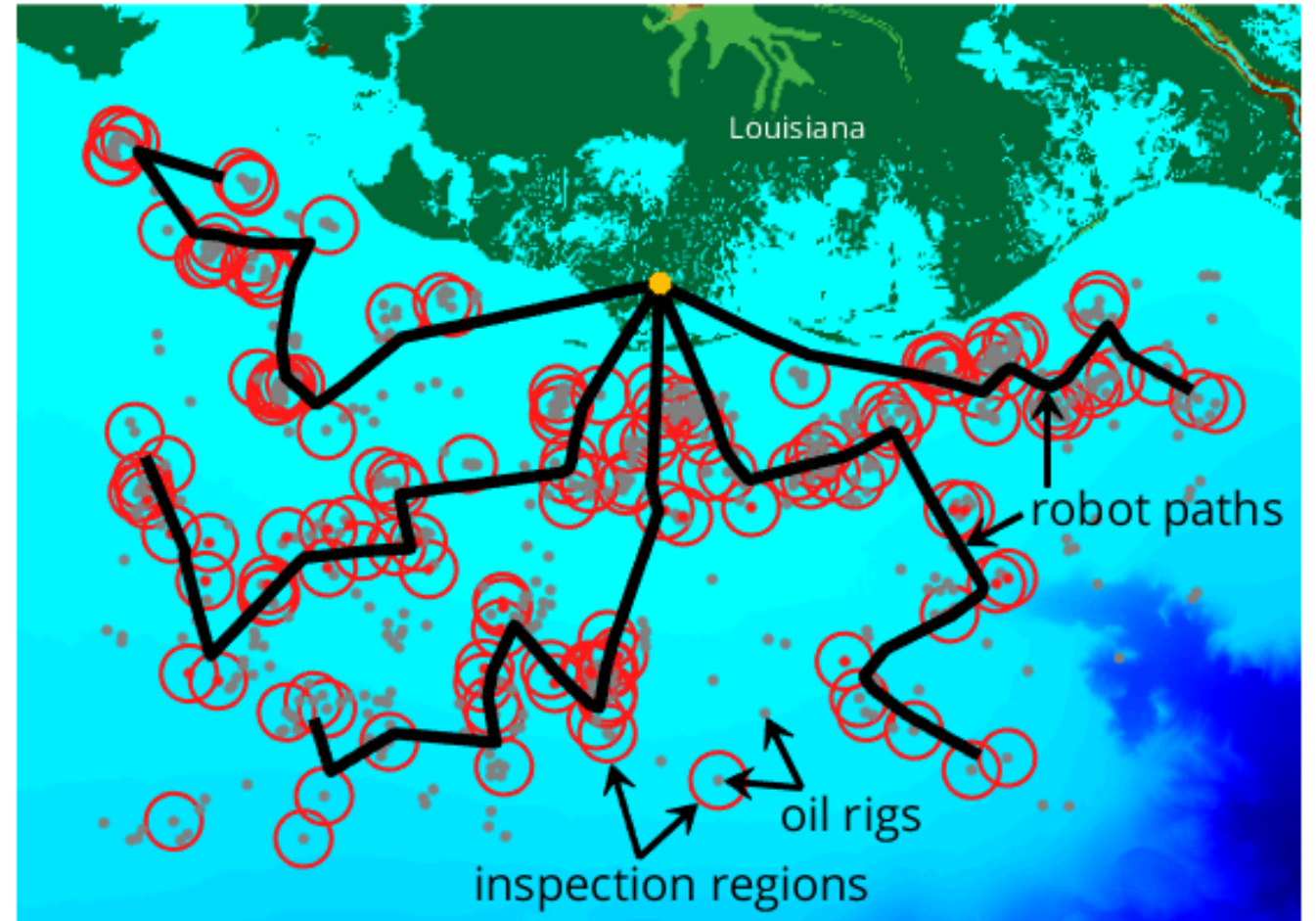
- Also see coverage planning: <https://fields2cover.github.io/>

Orienteering Problem



Problem:

Given a list of locations, the distances between locations, **and a maximum route length**, find a route that visits as many locations as possible?



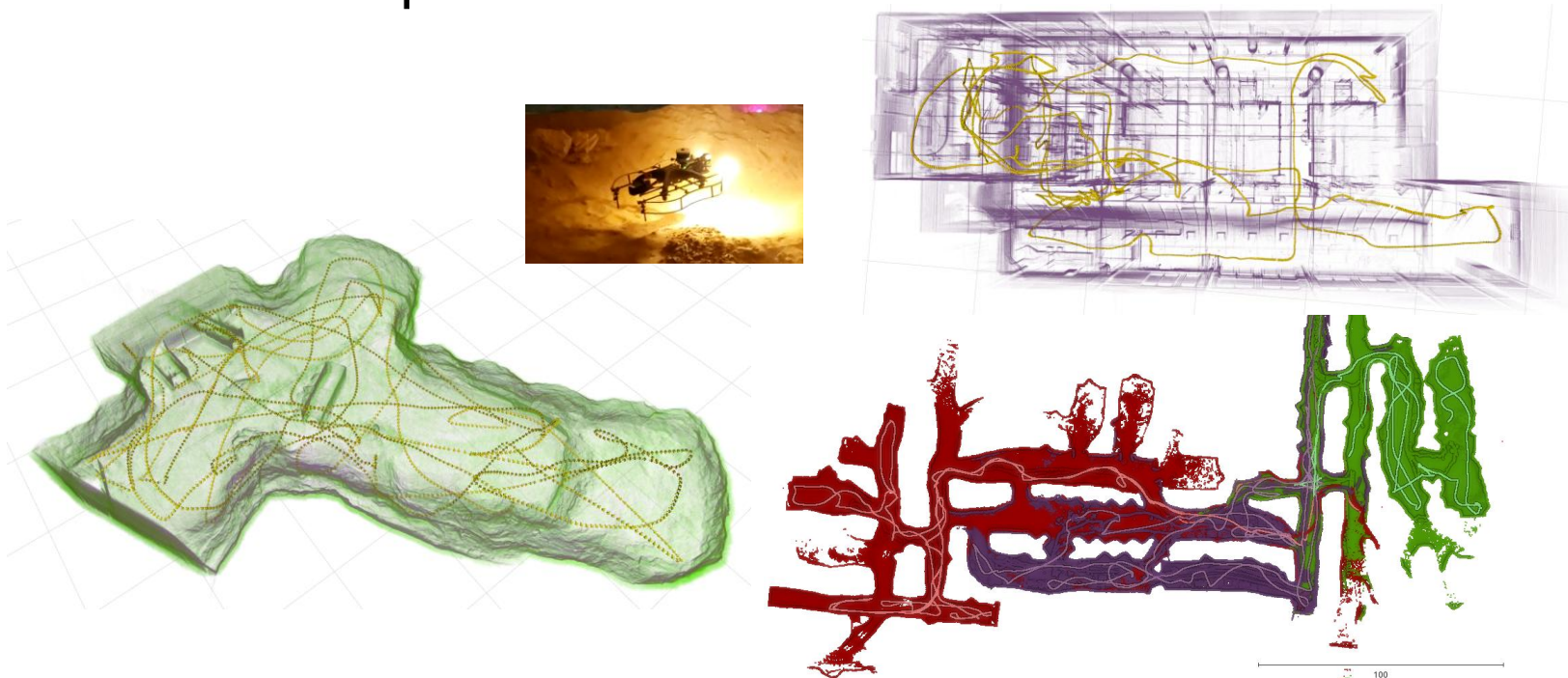


Exploration

Robot Exploration



- Aim: Construct a **map** of the world
- Classic robot planning problem
- Lots of practical uses

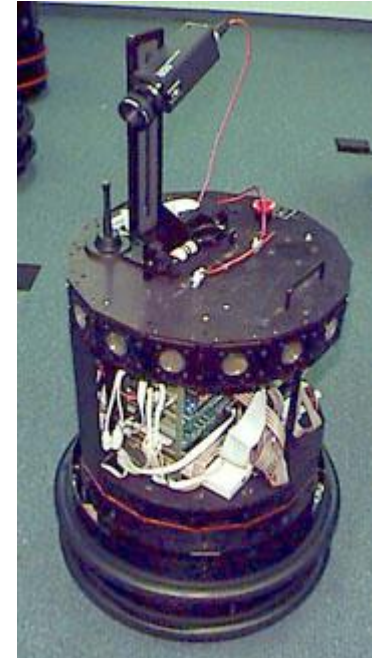
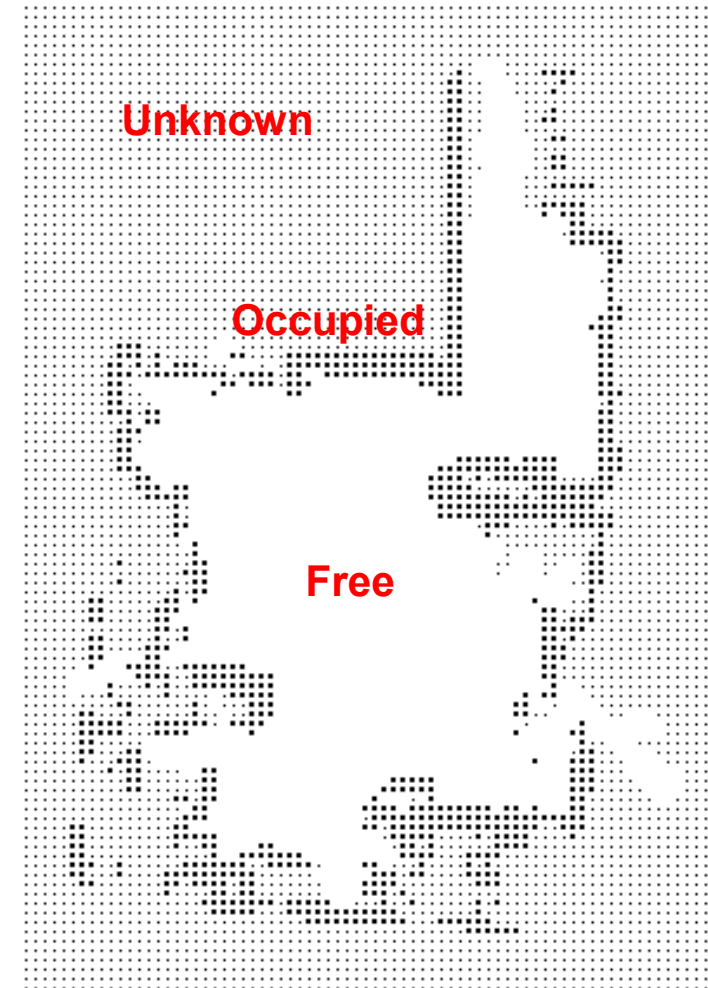


Problem Definition



Problem:

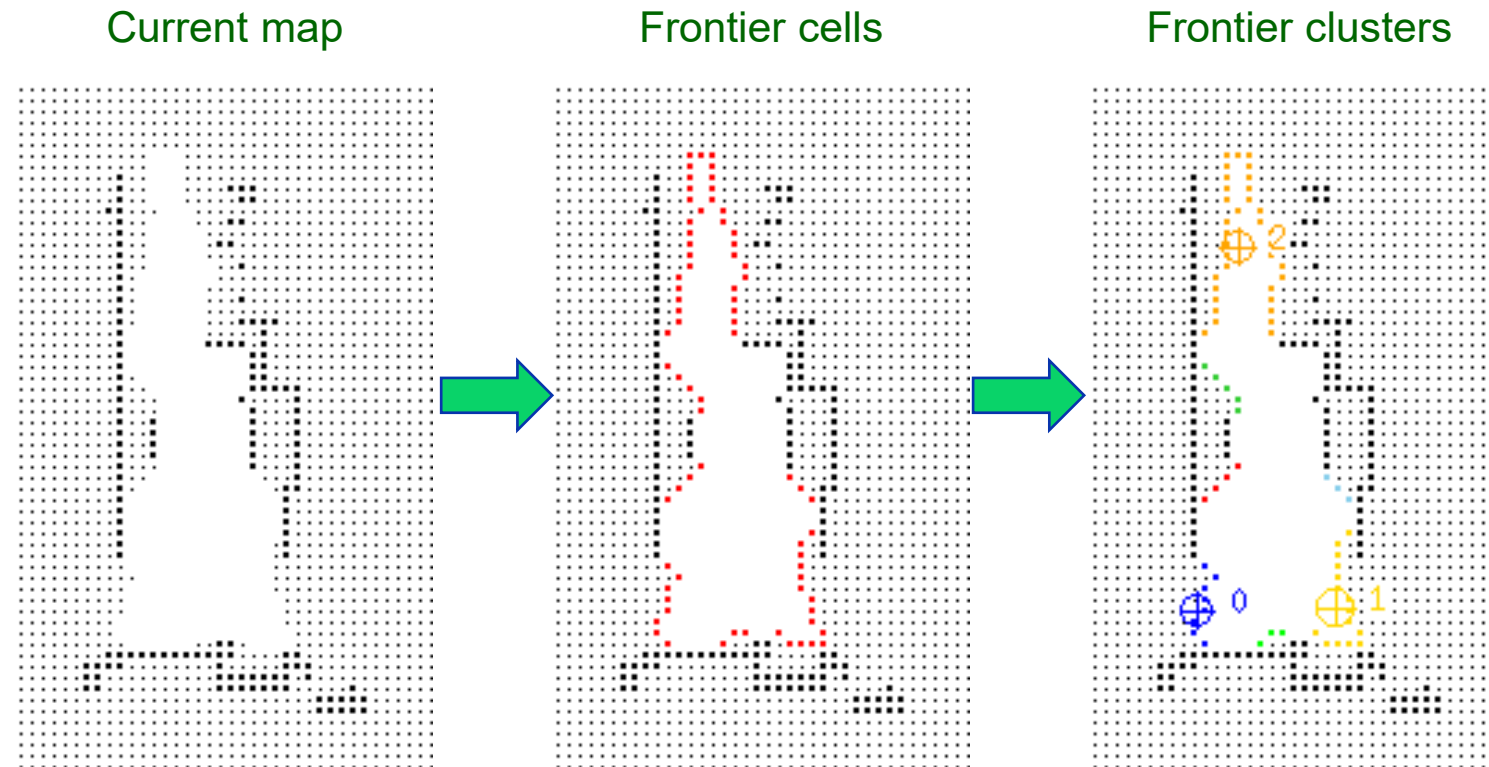
Given the current map of the world and a sensor model, where should a robot move to quickly build a full map of the world?



Frontier-Based Exploration

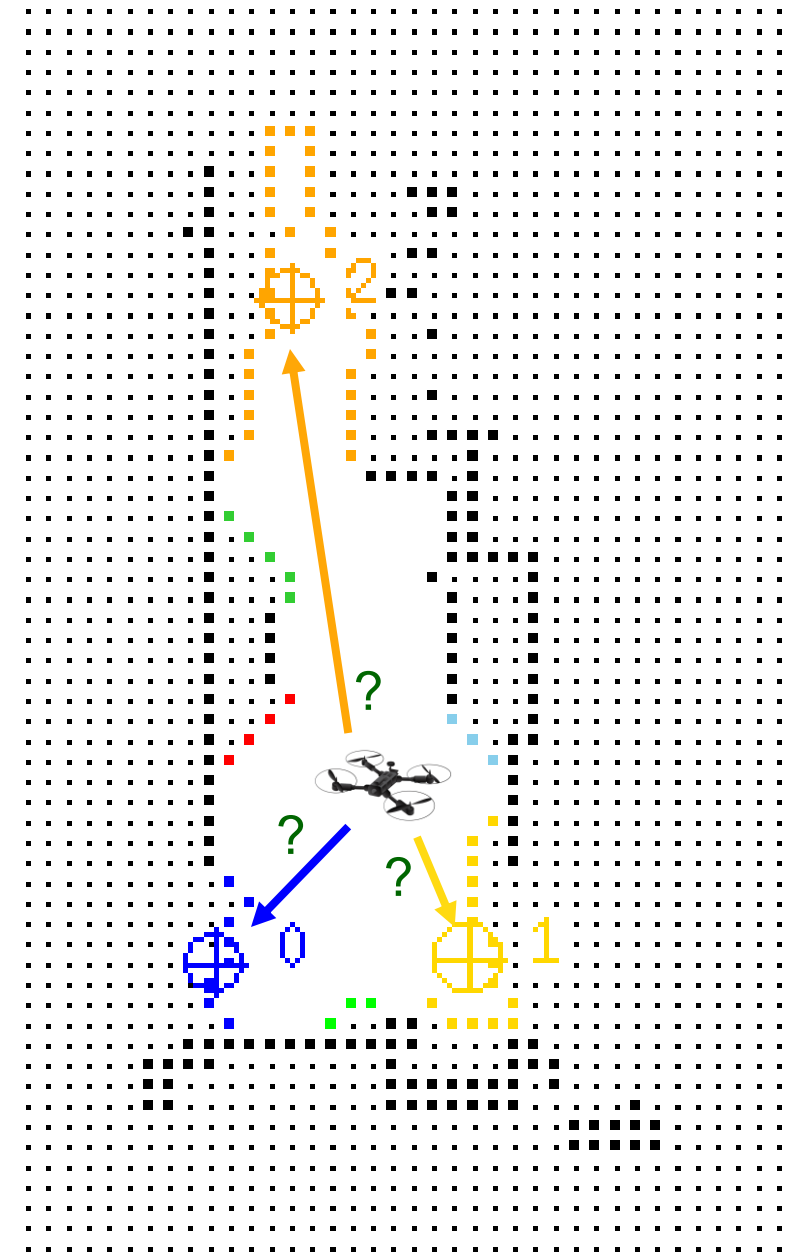


- **Key idea:** To gain the most new information, move to the boundary between free space and unknown space, i.e. “frontiers”

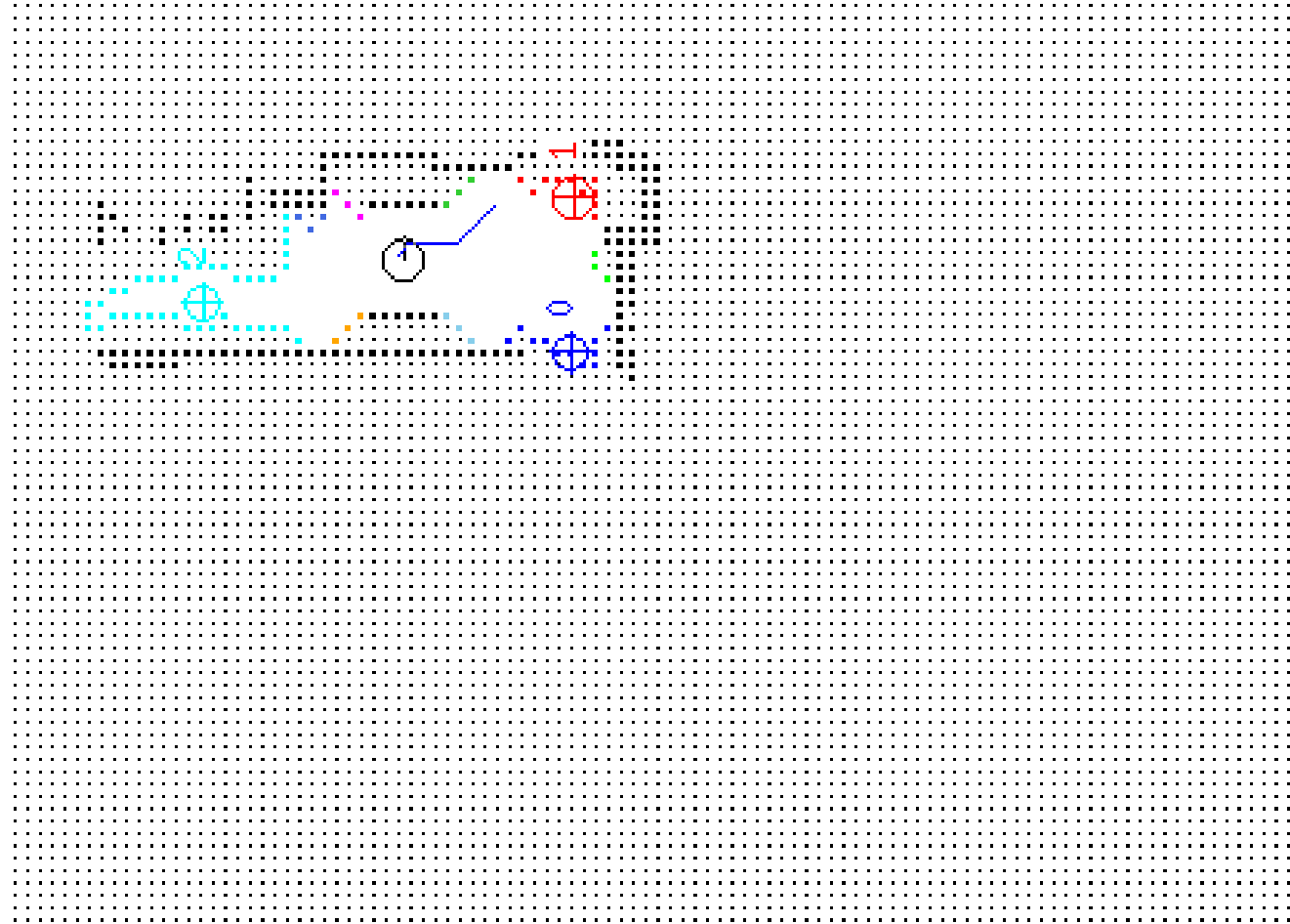


Frontier Selection Strategies

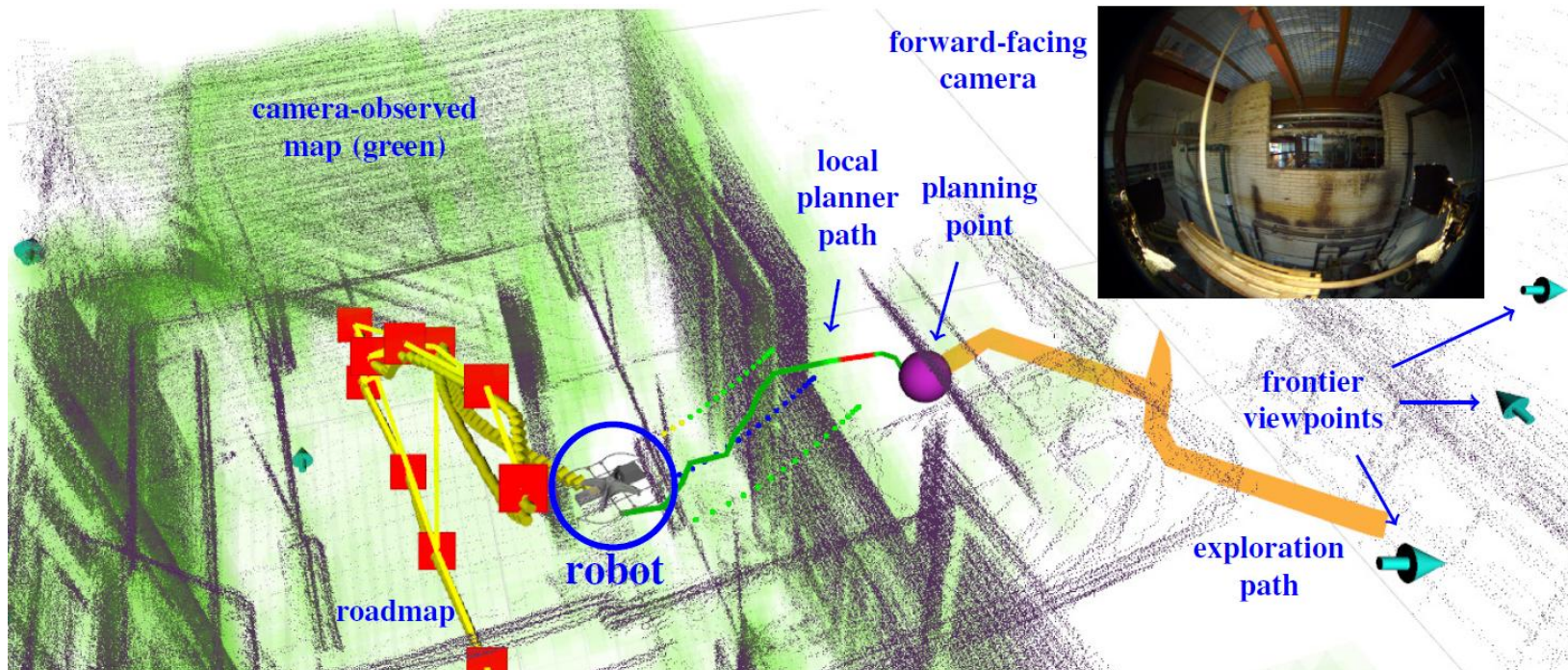
- Which frontier to go to first?
- Closest first
- Other quantities to consider:
 - Path distance
 - Safety
 - Frontier size
 - Information beyond the frontier
 - Coordination value
 - ...
 - Combinations of the above
- Then use path planning to selected frontier



Animation: Frontier-Based Exploration



Advanced Exploration



1. Frontier generation
 - a) Vision (coverage)
 - b) Range (exploration)
2. Viewpoint extraction
3. Viewpoint ranking
 - a) Distance
 - b) Momentum change
 - c) Coordination value
 - d) ...
4. RRT-Connect planner

Off-the-Shelf Packages



- You could implement solutions yourself
 - This would be challenging, but achievable for a 41068 project
- While there are no “default” solutions, there are many experimental packages:
 - I recommend trying this one first <https://github.com/robo-friends/m-explore-ros2> , which is referenced in the following instructions: <https://husarion.com/tutorials/ros2-tutorials/10-exploration/>
 - Lots of other variations on github, such as:
<https://github.com/AniArka/Autonomous-Explorer-and-Mapper-ros2-nav2>
https://github.com/SeanReg/nav2_wavefront_frontier_exploration?tab=readme-ov-file
 - It might take quite a bit of work to get these packages to do what you want!



Informative Path Planning

Informative Path Planning



- We have a **quantity of interest** that we wish to observe/estimate
 - A map
 - The location of an object
 - The type of an object
 - A temperature map of the ocean
 - Soil pH levels
- Maintain a probabilistic belief of the quantity of interest

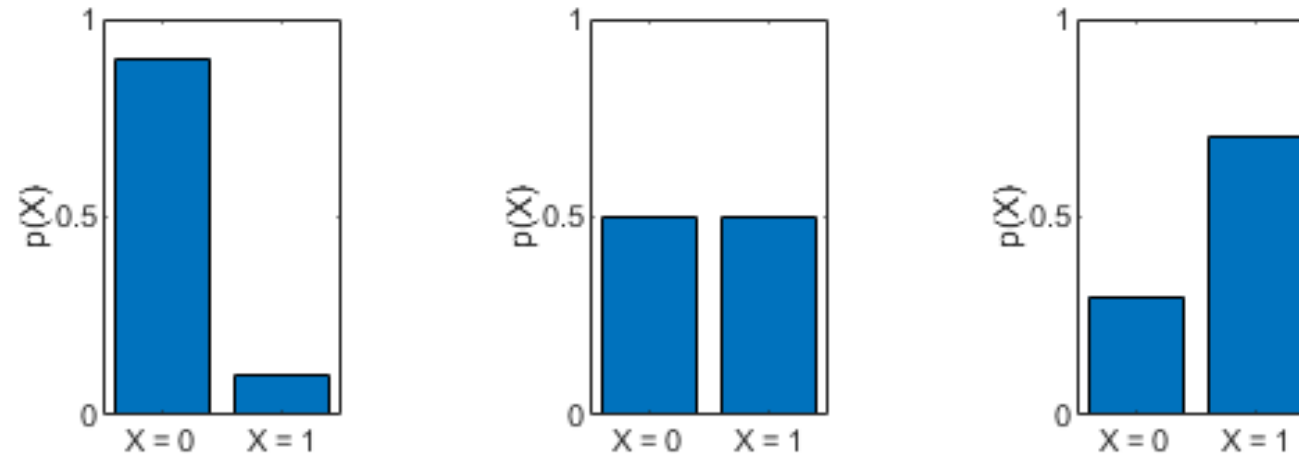
Problem:

Given the current belief of a quantity of interest,
where should a robot move to quickly
reduce the uncertainty of this belief?

Uncertainty: Example



- Define a random variable X
- Let $p(x_i)$ be the probability that $X = x_i$
- Assume X is a Bernoulli variable: i.e. can be either “0” or “1”



Which has the highest uncertainty?

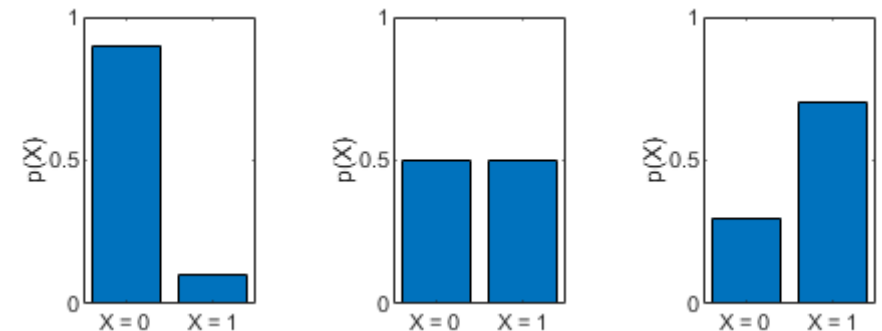
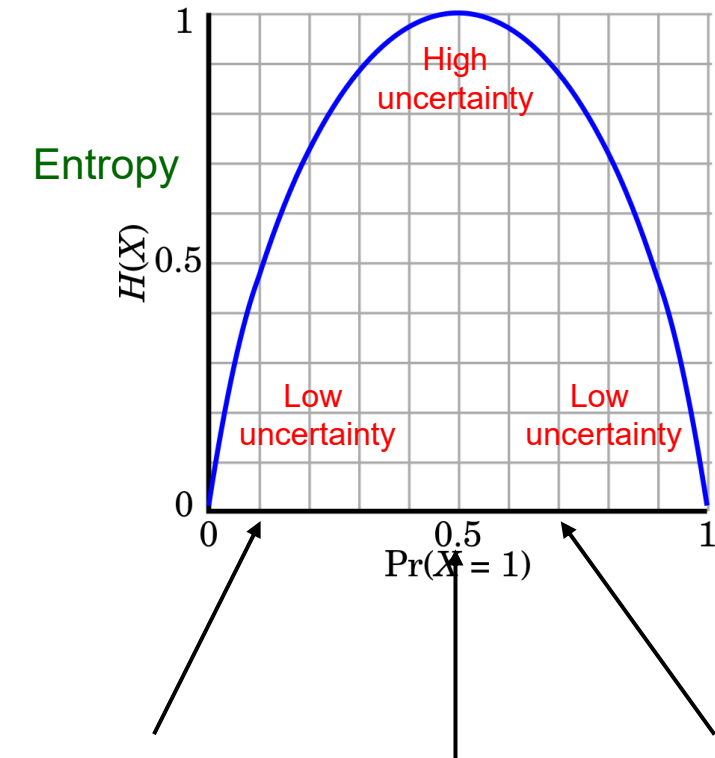
Uncertainty: Entropy

- We need a method to quantify “uncertainty”
- Entropy defined as:

$$H(X) = - \sum_{i=1}^n p(x_i) \log_b p(x_i)$$

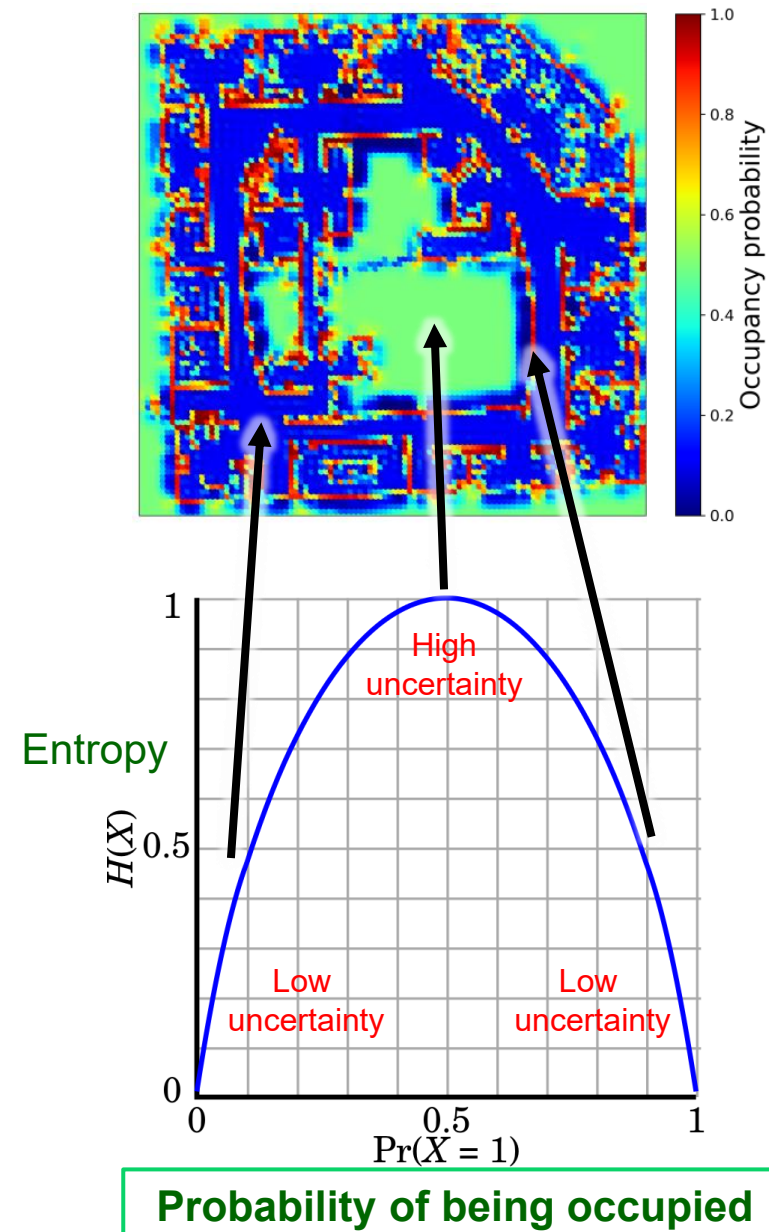
Sum over all possible outcomes for X

For a Bernoulli (0 or 1) distribution



Information-Based Exploration

- Alternative to frontier-based exploration
 - Mathematically correct
 - Much slower to compute
- Choose the next action that reduces “uncertainty”
- Use a probabilistic occupancy grid
- “Uncertainty” often defined as entropy

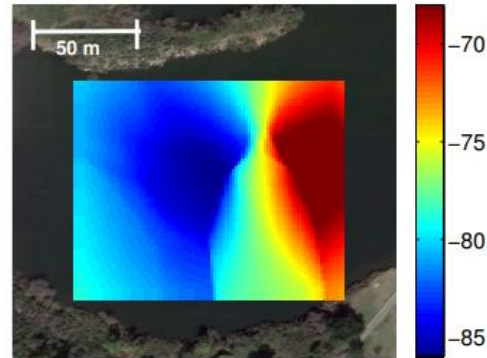


Estimate a Field



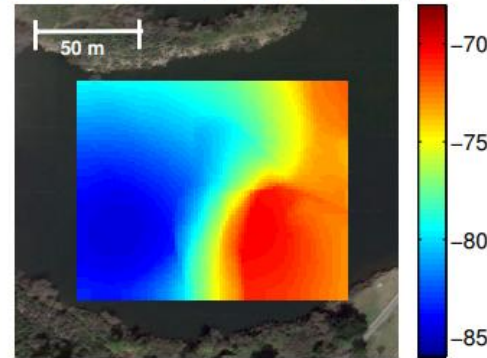
Estimate of wireless
signal strength

Predicted Wireless Signal Strength (dBm)



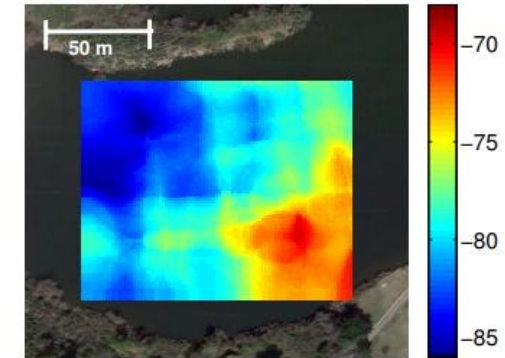
(a) After 400 m branch/bound path

Predicted Wireless Signal Strength (dBm)



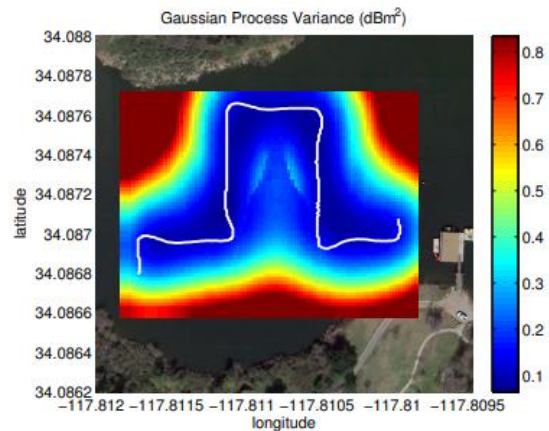
(b) After 400 m RIG-tree path

Predicted Wireless Signal Strength (dBm)

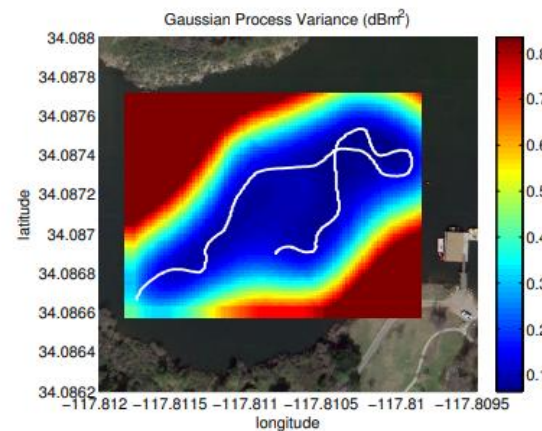


(c) After 2500 m survey

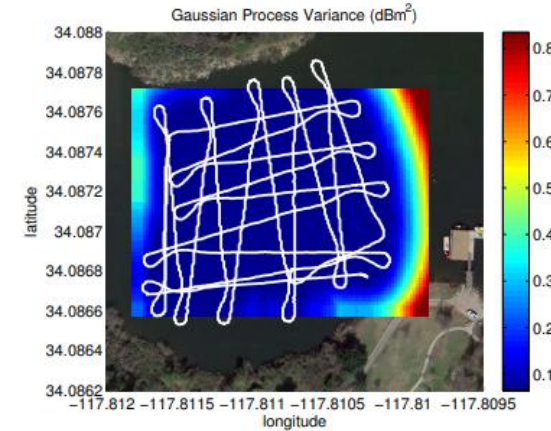
Uncertainty of
estimate



(d) After 400 m branch/bound path



(e) After 400 m RIG-tree path



(f) After 2500 m survey

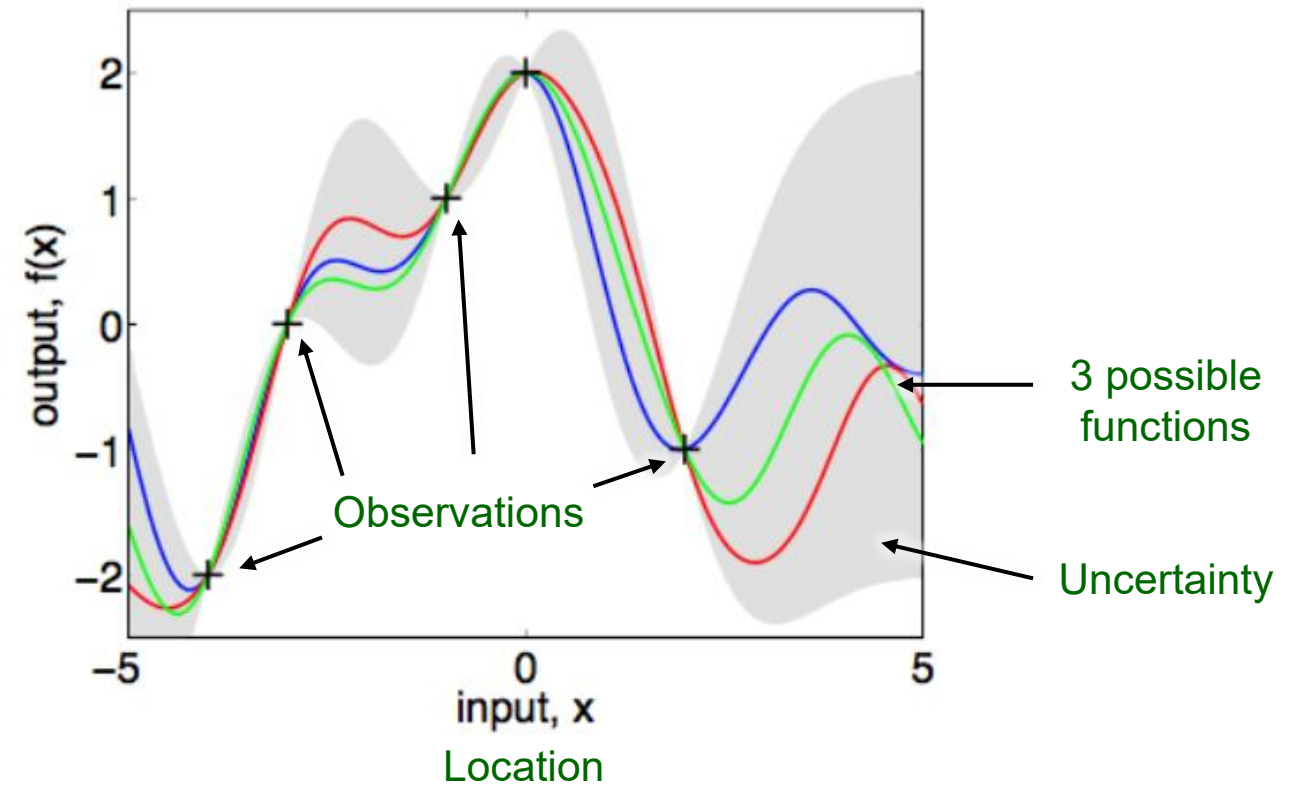
[Hollinger, 2014]

Gaussian Processes



- Gaussian Processes are probability distributions over functions
- Infinite dimensional Gaussian distribution
- A covariance function defines the relationship between points
- Off-the-shelf package:
https://scikit-learn.org/stable/modules/gaussian_process.html

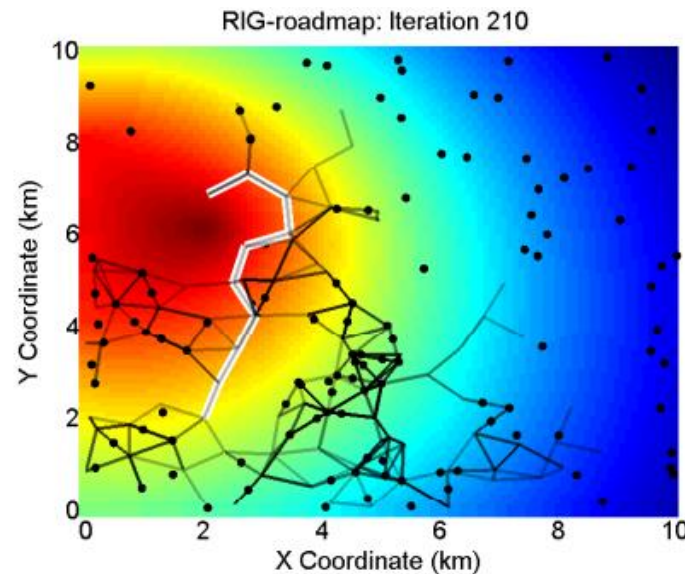
Quantity of
interest



Algorithms



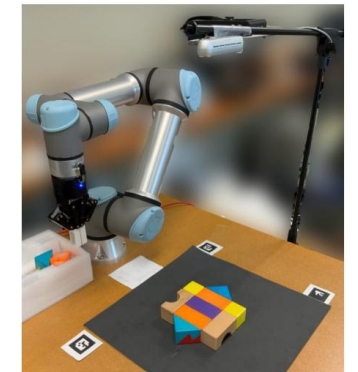
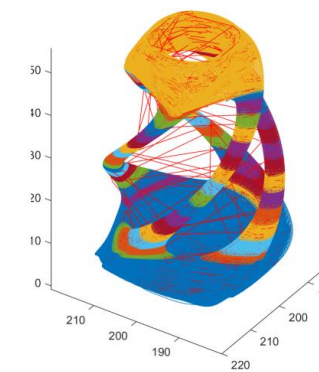
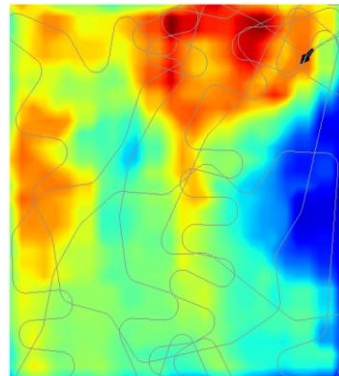
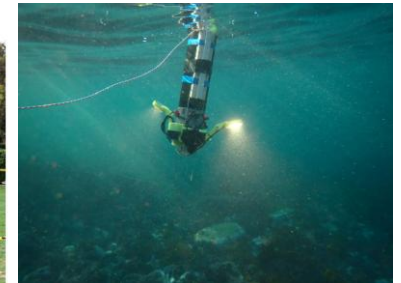
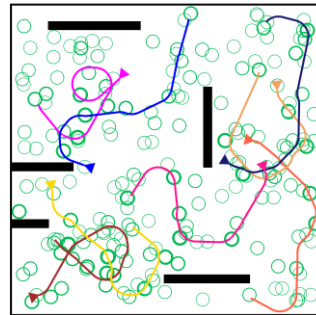
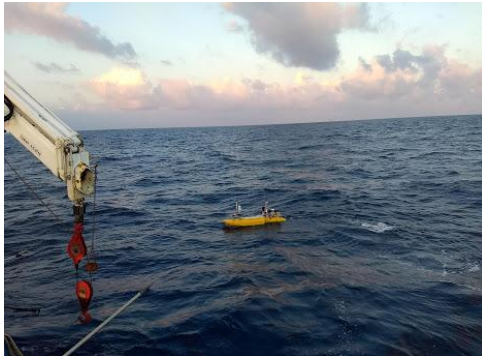
- **Greedy:** Move to the location that has the highest uncertainty
- **Branch and bound:** Tree search that iteratively bounds the solution quality
- **Sampling-based methods:** RRT over the information field
- **Monte Carlo tree search:** Next section!



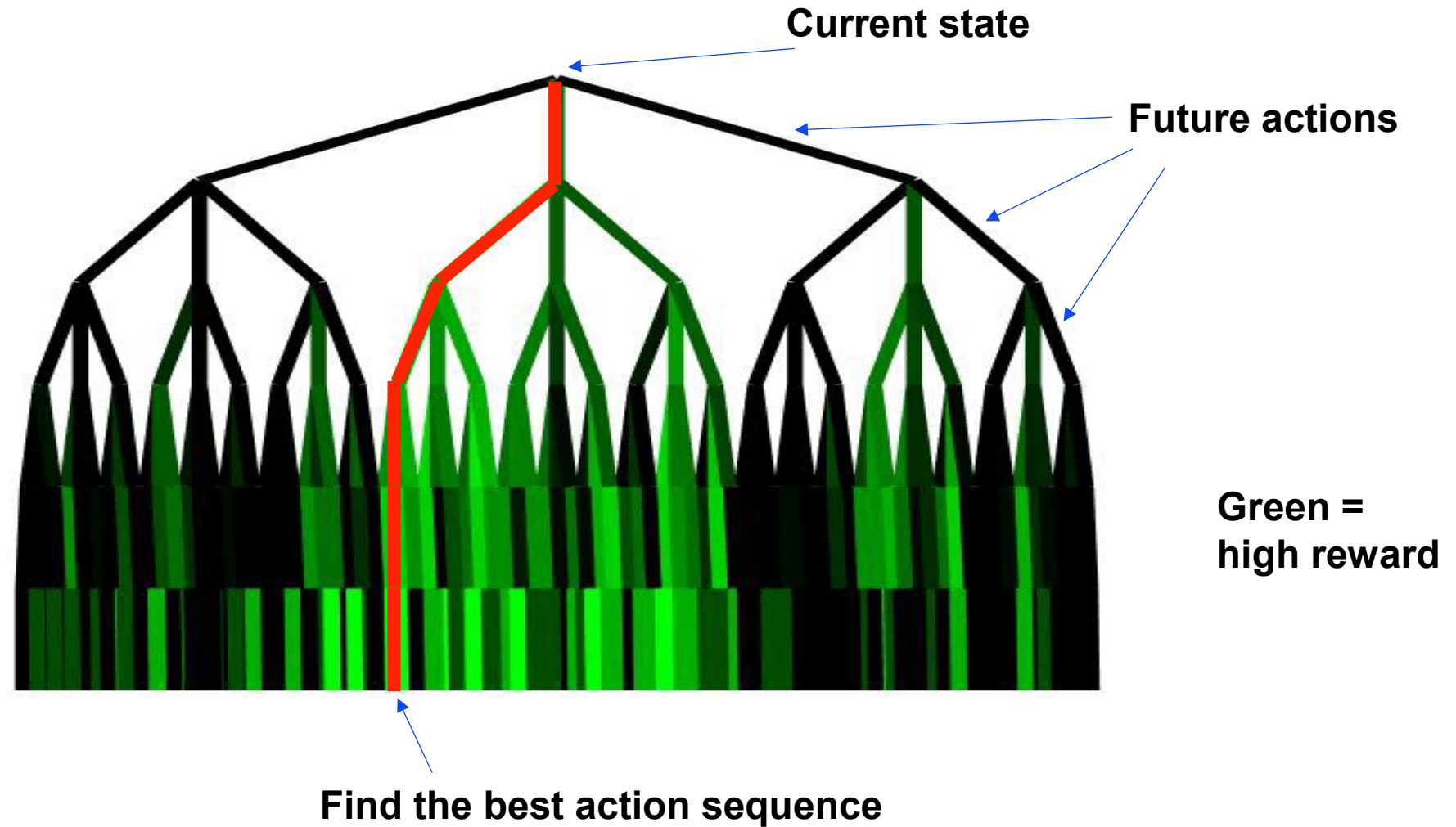


Monte Carlo Tree Search

MCTS for Robot Information Gathering



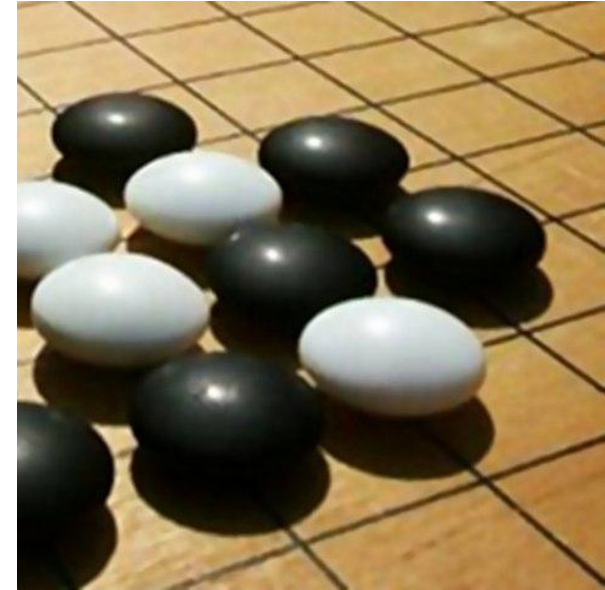
Tree Search Problem



Monte Carlo Tree Search (MCTS)

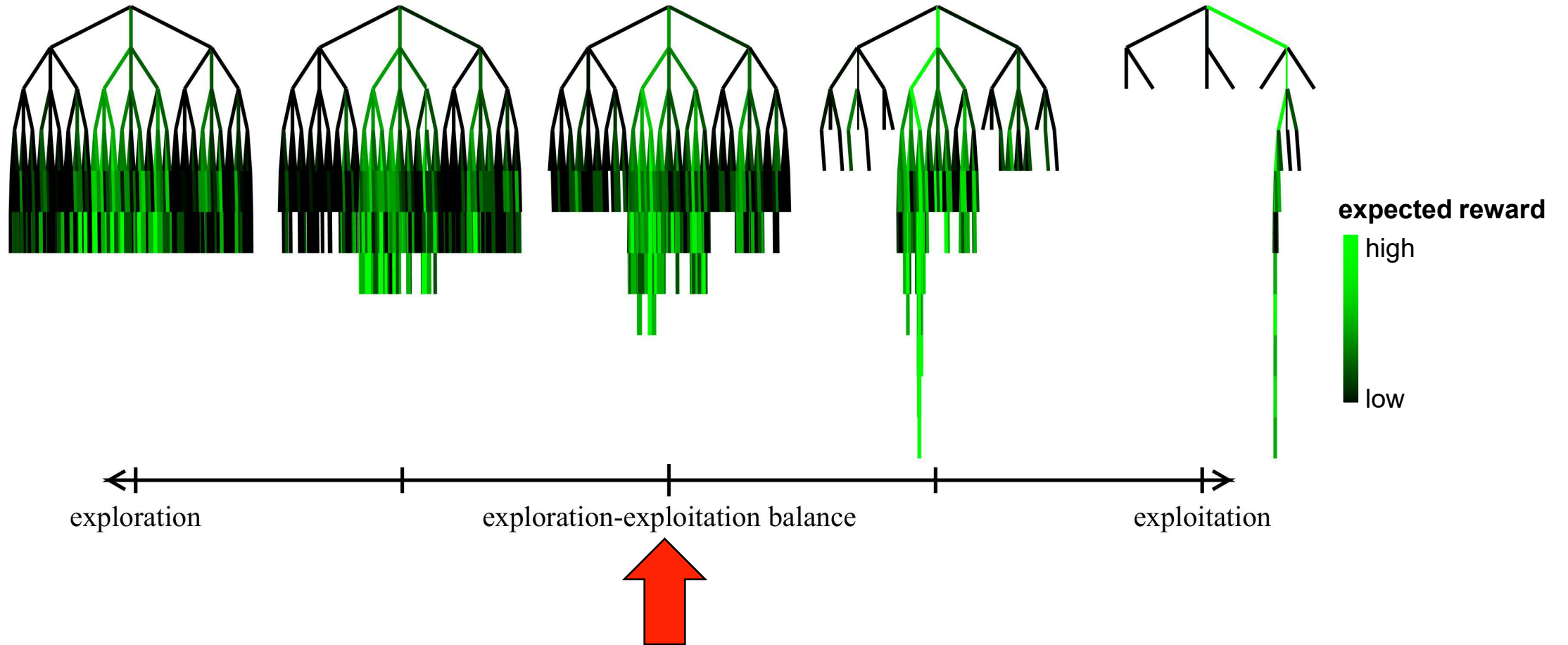


- *Biased* random sampling
- Exploits “smoothness” of search space
- Any-time
- Only requires evaluation of *full* paths
- Can incorporate problem-specific heuristics
- Has theoretical convergence guarantees
- **Recently become popular in robotics**

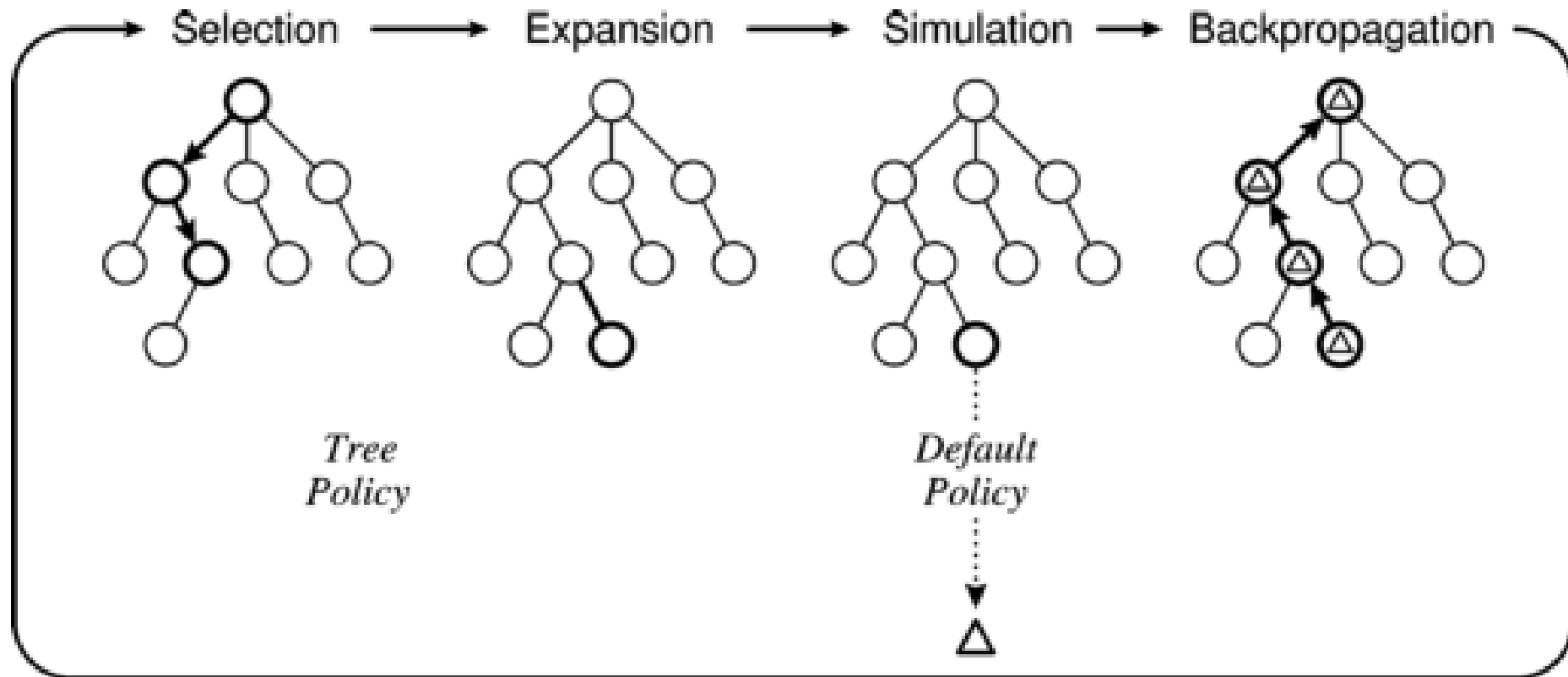


Computer Go [Silver, 2017]

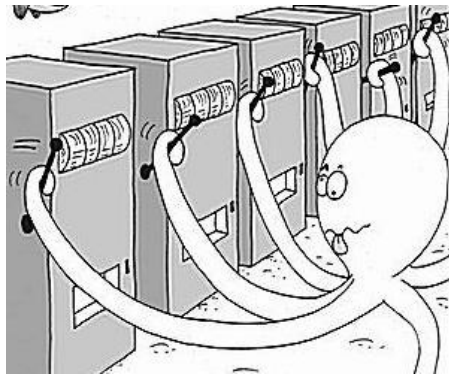
MCTS Behaviour



MCTS Algorithm



Upper-Confidence Bound (UCB)



Multi-armed bandit

Average rollout score

$$f(c) = W(\pi_c) + \sqrt{\frac{2 \ln n_v}{c_v}}$$

Upper confidence bound (UCB1)

Exploitation

Exploration

Parent #visits

Child #visits

MCTS in Action



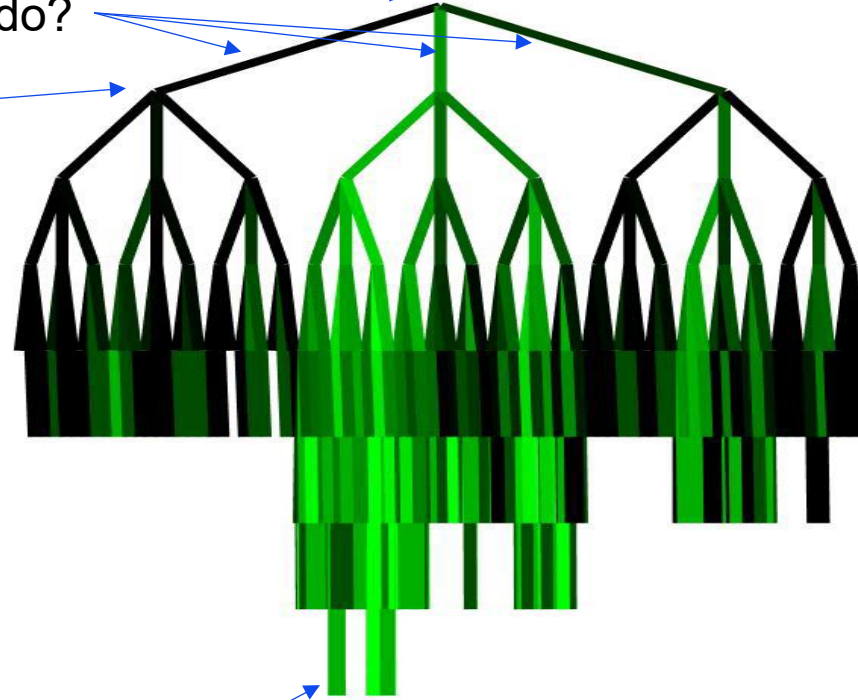
How To Implement MCTS for My Problem?



What is a **state**?

For each **state**, what **actions** can I do?

For each **action**, what is my **next state**?

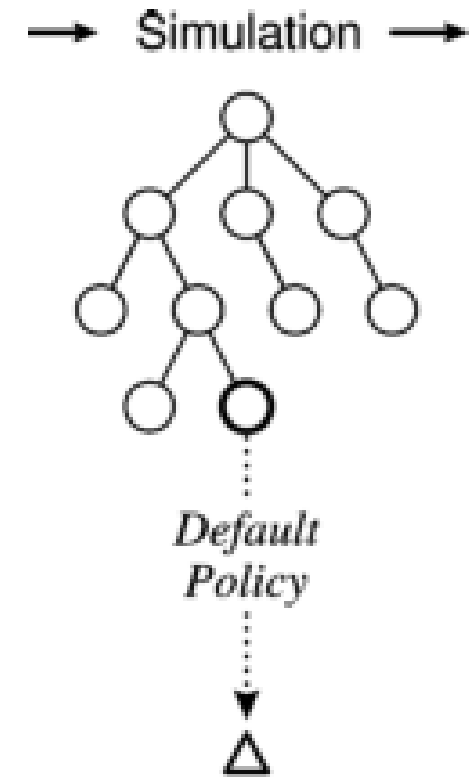


For each complete sequence of actions, what is its **reward**?
Scaled between roughly 0 and 1

Tuning Parameters



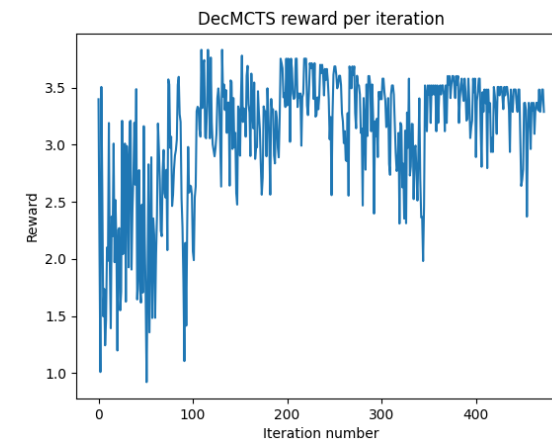
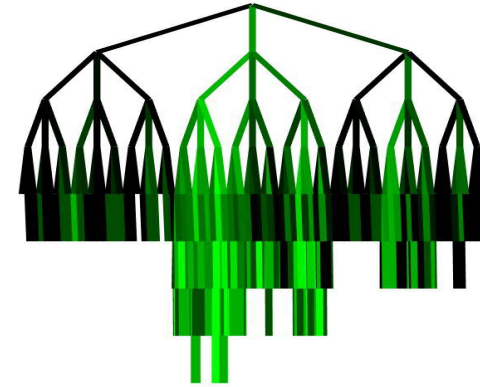
- Planning horizon (size of tree)
 - Random
 - Greedy
- Rollout policy
 - $\sqrt{2}$



Debugging Tips



- Observe the solutions
- Plot the tree
- Show empirically that it is converging
- Optimisation: Reward evaluation is the slowest part
 - Use a profiler
 - Caching where possible

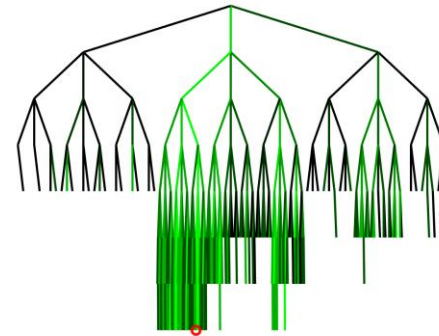


Dec-MCTS for Multi-Robot Coordination



Performed asynchronously
by each robot

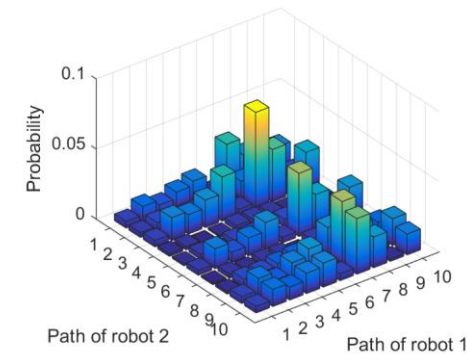
(a) Grow search tree for
own actions



(c) Communicate distributions
with other robots



(b) Decentralised optimisation of
probability distributions





Behaviour Trees

Multi-Task Missions



Subterranean Challenge

Exploring area



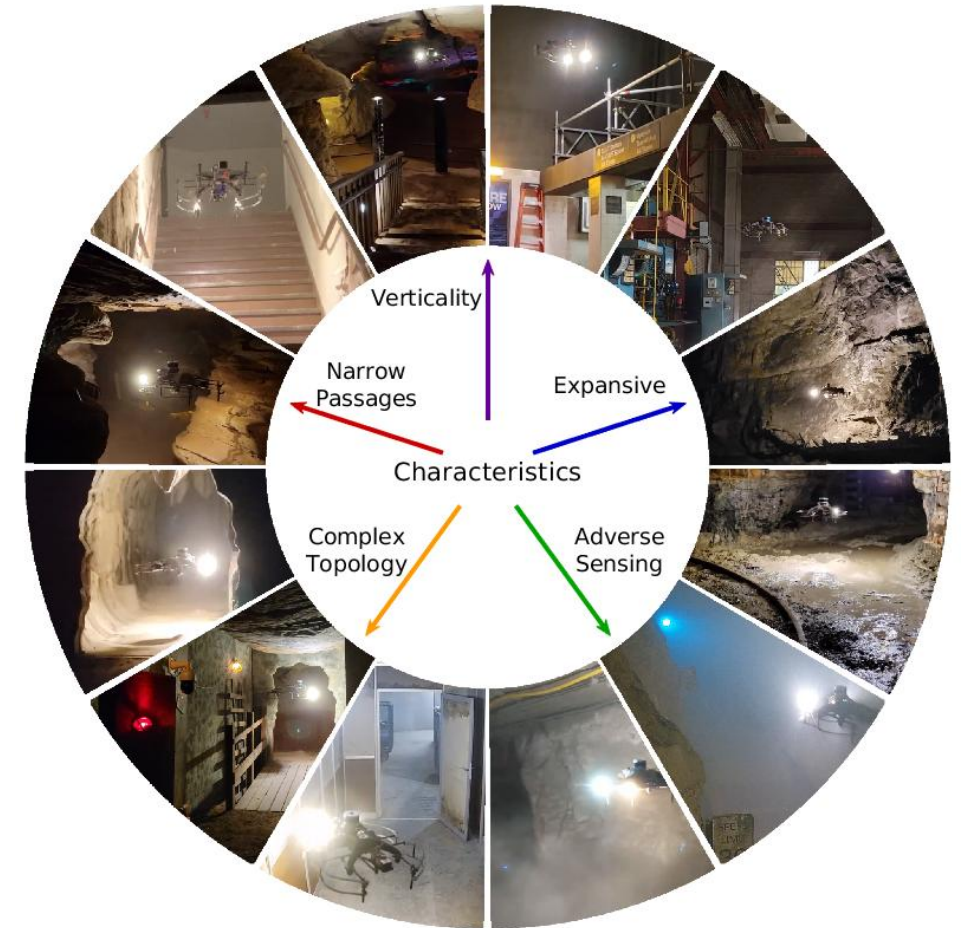
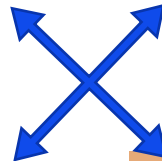
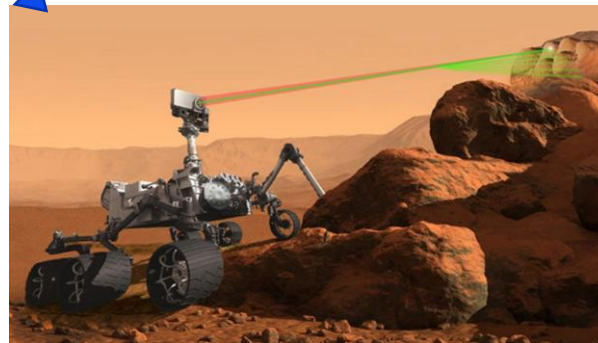
Avoid hazard



Drilling



Scan rocks



Behaviour Tree Definitions

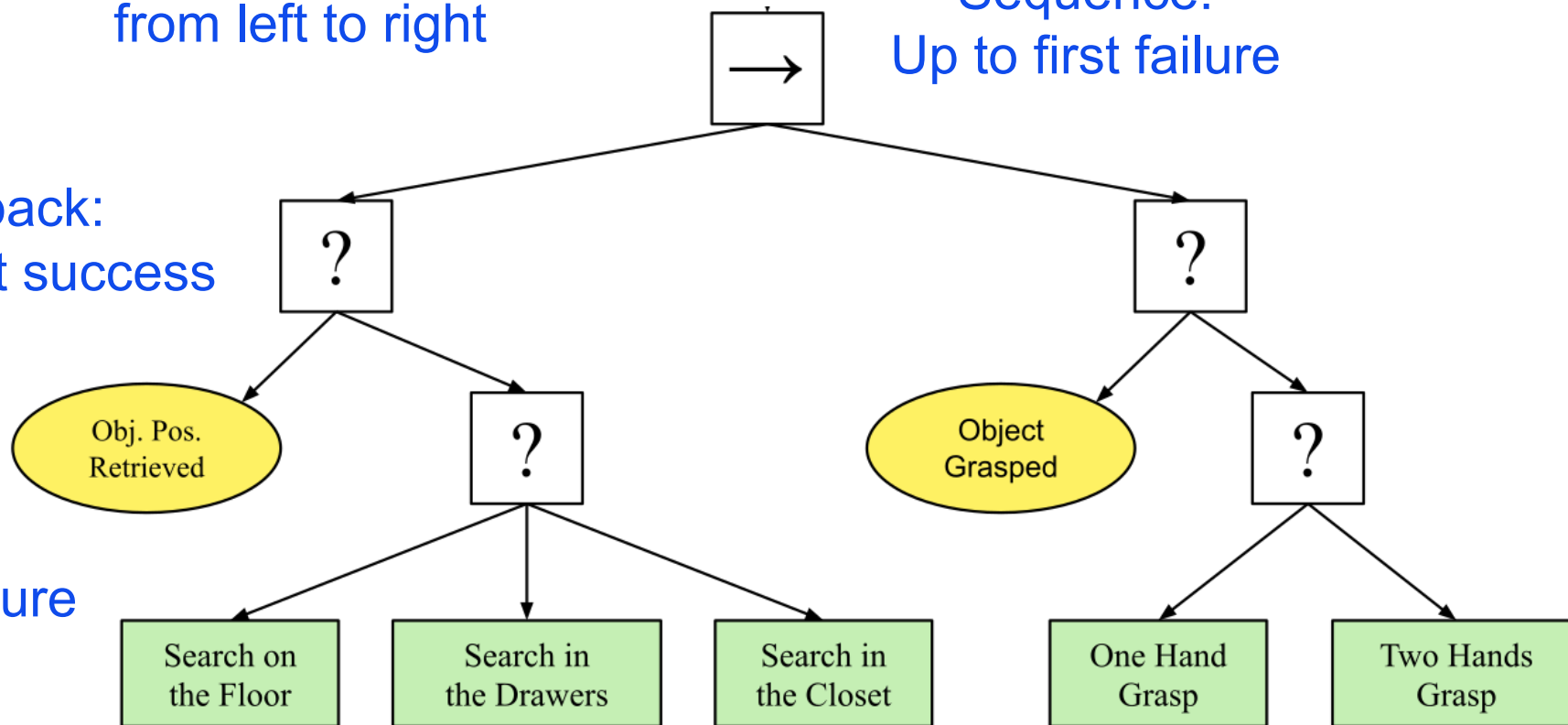


Activate recursively,
from left to right

Sequence:
Up to first failure

Fallback:
Up to first success

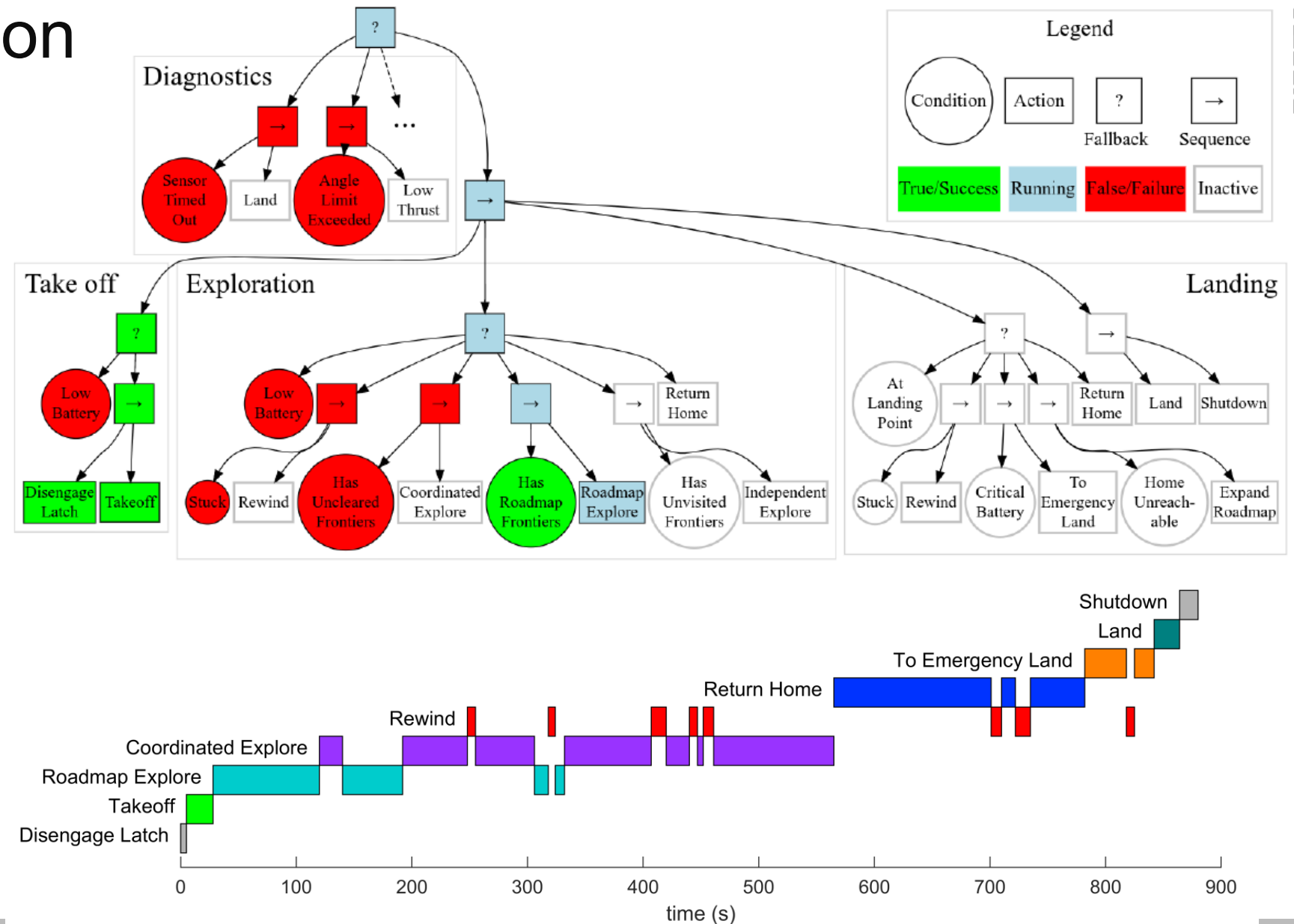
Conditions:
Success or failure



Actions:
Success or failure

... and other
types of nodes

Example Mission



- ## Nav2 behaviour tree





Reflection

Conclusions



- **Robot decision making** is the process of choosing *what the robot should do next* based on its current knowledge, goals, and environment.
- **Travelling salesman problem** aims to visit a set of locations as quickly as possible
- **Exploration** aims to build a map of an environment by visiting unobserved locations
- **Informative path planning** gathers the most information, as defined by an application
- **Monte Carlo tree search** is an algorithm for decision making
- **Behaviour trees** is a way of encoding high-level task switching

Questions for your 41068 Project



- Ideas for different types of decision making
- Your project must have some decision making
- How will your robot decide where to go next?
 - What is the objective?
 - What algorithms will solve this problem?
 - How will these algorithms connect to the rest of your system?

